

Правительство Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего профессионального образования
«Национальный исследовательский университет»
«Высшая школа экономики»
Нижегородский филиал

Факультет математики, информатики и компьютерных наук

КУРСОВАЯ РАБОТА

Применение LLM и Classical ML алгоритмов для решения задач анализа
уязвимостей в коде

Выполнил:

Студент 3 курса группы 23КНТ6

Антонов Артём Владимирович

Научный руководитель:

Приглашенный преподаватель

Воевоводкин Вадим Сергеевич

Нижний Новгород

Май 2026 г.

Содержание

1	Введение	3
2	Теоретические основы автоматизированного анализа уязвимостей в программном коде	5
2.1	Понятие уязвимости программного обеспечения	5
2.2	Методы и инструменты статического анализа кода (SAST)	8
2.2.1	Принципы работы статических анализаторов	8
2.2.2	Представления кода в виде графов	9
2.2.3	Классификация подходов к статическому анализу	10
2.2.4	Обзор популярных решений	12
2.2.5	Ограничения традиционных подходов	13
2.3	Применение методов машинного обучения в задачах кибербезопасности	14
2.3.1	Классическое машинное обучение для анализа кода	14
2.3.2	Глубокое обучение и графовые нейронные сети (GNN)	16
2.3.3	Большие языковые модели (LLM) в анализе уязвимостей	17
2.3.4	Сравнительный анализ и обоснование гибридного подхода	18
3	Разработка гибридной архитектуры сканера уязвимостей на основе ReGVD, LoRA и FAISS	21
3.1	Базовая архитектура: модель ReGVD	21
3.2	Модуль семантического поиска: Embedding Head + FAISS	23
3.3	Адаптивное дообучение: внедрение LoRA-адаптеров	26
3.4	Гибридная схема принятия решений	29
4	Экспериментальная оценка и анализ результатов	31
4.1	Датасеты и метрики оценки	31
4.1.1	Датасет Devign (CodeXGLUE Defect Detection)	31
4.1.2	Датасет MegaVul	32
4.1.3	Препроцессинг и формат данных	33
4.1.4	Классификационные метрики	33
4.1.5	Системные показатели и метрики энергоэффективности	34
4.1.6	Обоснование выбора метрик	35
4.2	Протокол экспериментов	35
4.2.1	Baseline-модели и сценарии сравнения	36

4.2.2	Конфигурация обучения и аппаратная среда	36
4.2.3	Протокол валидации и подбора гиперпараметров	37
4.2.4	Критерии успешности экспериментов	38
4.2.5	Протокол эксперимента по LLM-triage уязвимостей	38
4.3	Результаты и их интерпретация	40
4.3.1	Фаза 1: Воспроизведение ReGVD на Devign	41
4.3.2	Фаза 2: Гибридный пайплайн на Devign	41
4.3.3	Фаза 3: Zero-shot перенос гибридного пайплайна на MegaVul	42
4.3.4	Фаза 4: LoRA-адаптация гибридного пайплайна на MegaVul	42
4.3.5	Сводный анализ результатов всех фаз	43
4.3.6	Системная производительность и энергоэффективность	44
4.3.7	Результаты эксперимента по LLM-triage	45
5	Заключение	46
6	Приложение	49
6.1	Приложение А. Иллюстративные материалы и сводное сравнение подходов	49
6.2	Приложение Б. Шаблоны системных промптов LLM-triage	55
6.2.1	Системный промпт основного режима (SARIF-findings)	55
6.2.2	Системный промпт режима оценки (eval-mode)	56
6.2.3	Структура итогового JSON-ответа	57
	Список литературы	58

1 Введение

В условиях стремительной цифровизации экономических, государственных и технологических процессов программное обеспечение становится критическим элементом современной инфраструктуры, что сопровождается экспоненциальным ростом количества, масштаба кибератак [4, 5]. Несмотря на возрастающее внимание к информационной безопасности, многие организации сталкиваются с системным дефицитом ресурсов для оперативного мониторинга и устранения уязвимостей в исходном коде. В ответ на этот вызов в индустрии закрепляется тренд на интеграцию практик безопасности на ранних этапах жизненного цикла разработки, известный как подход shift-left и DevSecOps [20]. Своевременное выявление и исправление дефектов защищённости на стадиях проектирования и написания кода, а не на этапе предпродажного тестирования или эксплуатации, позволяет существенно снизить финансовые издержки, репутационные риски и время вывода продукта на рынок [2]. Ключевым инструментарием реализации данной парадигмы выступают автоматизированные сканеры уязвимостей и системы статического анализа кода, однако традиционные решения, базирующиеся преимущественно на сигнатурном поиске, регулярных выражениях и жёстких правилах, демонстрируют ограниченную адаптивность к новым классам уязвимостей, сложным логическим ошибкам и динамично обновляемым фреймворкам [16, 3]. Одновременно объёмы генерируемого и поддерживаемого исходного кода многократно превышают аналитические возможности специалистов. Человеческий фактор в условиях информационной перегрузки становится критическим ограничением, поскольку экспертам физически сложно отслеживать все паттерны, особенно в распределённых и быстро меняющихся репозиториях, что неизбежно приводит к пропуску скрытых зависимостей, каскадных дефектов и накоплению технического долга безопасности [15, 13]. Перспективным вектором преодоления указанных ограничений выступает применение методов искусственного интеллекта, в частности алгоритмов машинного обучения и больших языковых моделей. Классические ML-алгоритмы и графовые нейронные сети демонстрируют высокую скорость и воспроизводимость при работе со структурированными представлениями кода, эффективно выявляя статистические аномалии и снижая уровень ложных срабатываний при корректной инженерии признаков [3]. Большие языковые модели, благодаря глубокому контекстному представлению семантики программных текстов и способности к логическому рассуждению, позволяют анализировать нетривиальные взаимосвязи, интерпретировать намерения разработчика и формировать объяснимые рекомендации по исправлению [23, 21]. Однако практическая интеграция исключительно генеративных моделей в production-сканеры ограничена высокой латентностью, вычислительной стоимостью и вероятностной природой выводов. В связи с этим актуальным направлением становится проектирование гибридных архитектур, сочетающих структурный анализ на базе графовых сетей,

параметро-эффективные методы адаптации и семантический поиск для реакции на новые паттерны [1, 17, 18]. Подобная синергия позволяет автоматизировать не только детекцию, но и первичный триаж находок, приоритезируя потенциально опасные фрагменты по степени уверенности и семантической близости к известным эксплойтам [14, 27]. Исходя из обозначенных проблем, целью настоящей курсовой работы является исследование архитектурных и алгоритмических возможностей интеграции графовых нейронных сетей, параметро-эффективных методов адаптации и векторного поиска для повышения точности, полноты и интерпретируемости автоматизированного анализа уязвимостей в исходном коде, а также проектирование концепции гибридного сканера, объединяющего преимущества структурного обобщения и семантической аналогии. Для достижения поставленной цели в работе последовательно рассматриваются существующие подходы к статическому анализу и применению машинного обучения в кибербезопасности — с акцентом на их практических ограничениях. Отдельно изучается теоретическое обоснование использования моделей семейства ReGVD, адаптации через LoRA и библиотек приближённого поиска, после чего предлагается многоуровневая архитектура гибридного сканера. В основе реализованной системы лежит двухэтапная логика обработки: на первом этапе графовая нейронная сеть выполняет первичный отбор потенциально уязвимых фрагментов, принимая во внимание как локальные последовательностные паттерны, так и глобальные структурные связи. На втором этапе для каждого фрагмента строятся семантические эмбединги посредством контрастивного обучения; полученные векторы индексируются, что позволяет находить схожие случаи практически мгновенно — без необходимости переобучать базовую модель. Помимо этого, в систему введён механизм взвешенного объединения двух сигналов: вероятности, выдаваемой классификатором, и доли уязвимых соседей. Совместное использование этих оценок даёт возможность расставлять приоритеты среди найденных проблем, тем самым снижая операционную нагрузку на специалистов по безопасности. Архитектура поддерживает двухрежимную стратегию адаптации: оперативное “горячее обновление” индекса при поступлении единичных новых образцов и периодическое дообучение низкопроанговых адаптеров (LoRA) с использованием replay-буфера для сохранения знаний о ранее изученных паттернах. Объектом исследования выступают процессы автоматизированного обнаружения, классификации и верификации уязвимостей в программном коде, а предметом — алгоритмы машинного обучения, графовые нейронные сети и методы семантического поиска как инструментарий повышения эффективности анализа. Практическая значимость работы заключается в возможности применения предложенной гибридной архитектуры в корпоративных конвейерах непрерывной интеграции для раннего выявления дефектов безопасности, сортировки предупреждений и адаптации к новым классам угроз с минимальными вычислительными затратами, что соответствует требованиям к устойчивости экосистем и оптимизации ресурсов команд разработки и информационной безопасности.

2 Теоретические основы автоматизированного анализа уязвимостей в программном коде

2.1 Понятие уязвимости программного обеспечения

В современном контексте информационной безопасности термин *уязвимость программного обеспечения* (software vulnerability) имеет строгое нормативное определение. Согласно Национальному институту стандартов и технологий США (NIST), уязвимость представляет собой *слабость в информационной системе, процедурах её безопасности, внутреннем контроле или из реализации, которая может быть использована источником угрозы для нарушения политики безопасности системы* [4]. Аналогичное определение приводит организация OWASP (Open Web Application Security Project), акцентируя внимание на том, что уязвимость — это дефект проектирования, реализации или конфигурации, создающий возможность для несанкционированного доступа, модификации данных или отказа в обслуживании [5].

В российской нормативно-правовой базе понятие уязвимости закреплено в ГОСТ Р 56939-2016 «Защита информации. Обеспечение безопасности при разработке программного обеспечения. Термины и определения». Согласно данному стандарту, уязвимость программы для ЭВМ — это *недостаток в программе для ЭВМ, который может быть использован для реализации угроз безопасности информации* [6]. Важным аспектом отечественного подхода является обязательная регистрация уязвимостей в базе данных ФСТЭК России (БДУ), что обеспечивает централизованный учёт и оперативное информирование субъектов критической информационной инфраструктуры.

Классификация уязвимостей представляет собой многоуровневую таксономию, позволяющую систематизировать дефекты по различным признакам. Наиболее распространённой является классификация по типу слабости, реализуемая через реестр CWE (Common Weakness Enumeration). Данный каталог, поддерживаемый сообществом MITRE, содержит более 900 типовых слабостей, сгруппированных в иерархическую структуру [7]. К наиболее критичным категориям относятся:

- **CWE-787: Out-of-bounds Write** — запись за границы выделенной области памяти, приводящая к переполнению буфера. Данная уязвимость лежит в основе большинства эксплойтов для языков C/C++ и позволяет злоумышленнику выполнить произвольный код в контексте уязвимого процесса [12].
- **CWE-89: SQL Injection** — внедрение вредоносных SQL-команд через пользовательский ввод. Уязвимость характерна для веб-приложений и позволяет обходить аутентификацию, извлекать конфиденциальные данные или модифицировать структуру базы данных [13].

- **CWE-79: Cross-site Scripting (XSS)** — внедрение клиентских скриптов в страницы, просматриваемые другими пользователями. Позволяет похищать сессионные токены, перенаправлять трафик или выполнять действия от имени жертвы [14].
- **CWE-416: Use After Free** — обращение к памяти после того, как соответствующая область уже была освобождена. Характерна прежде всего для системного ПО на C/C++: результатом становится неопределённое поведение программы, которое при определённых условиях позволяет атакующему выполнить произвольный код [15].
- **CWE-362: Concurrent Execution using Shared Resource with Improper Synchronization (Race Condition)** — состояние гонки, складывающееся при отсутствии надлежащей синхронизации между параллельными потоками. Последствия варьируются от повреждения данных до обхода механизмов контроля доступа [2].

Отдельным критерием классификации служит вектор атаки, который определяет, при каких условиях уязвимость может быть проэксплуатирована:

- *Локальные уязвимости* предполагают, что атакующий уже располагает учётной записью на целевой системе и возможностью запускать код. Показательный пример — эскалация привилегий через брешь в ядре ОС.
- *Удалённые уязвимости* могут быть эксплуатированы через сеть без предварительной аутентификации. Пример: переполнение буфера в сетевом демоне.
- *Уязвимости, требующие аутентификации*, занимают промежуточное положение: злоумышленник должен обладать легитимными учётными данными, но после входа может получить повышенные привилегии или доступ к данным других пользователей.

Для количественной оценки критичности уязвимостей применяется международная система CVSS (Common Vulnerability Scoring System), текущая версия 4.0 которой была опубликована в 2023 году консорциумом FIRST [8]. CVSS представляет собой векторную метрику, рассчитываемую на основе трёх групп параметров:

1. **Base Metrics** — базовые характеристики, не зависящие от времени и окружения:
 - *Attack Vector (AV)*: сетевой, смежный, локальный, физический;
 - *Attack Complexity (AC)*: низкая или высокая сложность атаки;
 - *Privileges Required (PR)*: отсутствие, низкие или высокие привилегии;
 - *User Interaction (UI)*: требуется или не требуется взаимодействие пользователя;

- *Scope (S)*: ограниченный или изменённый (влияние на другие компоненты);
- *Confidentiality/Integrity/Availability Impact (C/I/A)*: степень воздействия на конфиденциальность, целостность и доступность.

2. **Threat Metrics** (ранее Temporal) — характеристики, зависящие от времени: наличие публичного эксплойта, уровень зрелости средств исправления, степень подтверждения уязвимости.

3. **Environmental Metrics** — параметры, учитывающие специфику развёртывания: важность затронутых активов, наличие компенсирующих контролей, требования регуляторов.

Расчёт итогового скоринга производится по формуле, учитывающей взвешенную комбинацию метрик. Результирующее значение находится в диапазоне от 0.0 до 10.0 и интерпретируется следующим образом:

Помимо CVSS, для приоритизации реагирования используется каталог KEV (Known Exploited Vulnerabilities), поддерживаемый Агентством кибербезопасности и инфраструктурной безопасности США (CISA) [9]. Данный реестр содержит уязвимости, для которых зафиксированы случаи активной эксплуатации в реальных атаках. Включение уязвимости в KEV является сигналом для немедленного применения патчей, независимо от формального CVSS-скоринга.

В российской практике дополнительно применяется система классификации ФСТЭК России, которая учитывает не только технические параметры уязвимости, но и её влияние на объекты критической информационной инфраструктуры (КИИ). Уязвимости, затрагивающие системы КИИ, подлежат обязательной регистрации и контролю в соответствии с приказами ФСТЭК № 239 и № 250 [10, 11].

Важным аспектом современной практики является понимание того, что уязвимости не существуют изолированно. Ряд исследований фиксирует закономерность: дефекты кода нередко выстраиваются в каскадные цепочки, где несколько относительно безобидных слабостей — скажем, отсутствие валидации входных данных в сочетании с некорректной обработкой ошибок — совместно создают почву для высокоуровневой атаки [20]. Этот феномен, закрепившийся в литературе под названием *vulnerability chaining*, существенно осложняет автоматизированное обнаружение уязвимостей: недостаточно анализировать отдельные фрагменты кода — необходимо отслеживать их взаимодействие в масштабе всей программы.

Не менее значимой проблемой оказывается контекстная зависимость уязвимостей. Один и тот же паттерн кода способен быть как совершенно безопасным, так и критически опасным — в зависимости от среды выполнения, конфигурации системы и истории предшествующих операций [21]. Отсюда вытекает потребность в семантически насыщенных моделях анализа, которые

принимает во внимание не только синтаксическую структуру, но и поведенческие свойства кода. Именно это соображение во многом и обуславливает растущий интерес к интеграции методов машинного обучения и больших языковых моделей в задачи детекции уязвимостей [3, 23].

Таким образом, понятие уязвимости программного обеспечения представляет собой многогранную категорию, требующую учёта нормативных определений, таксономических классификаций, метрик критичности и контекстных зависимостей. Глубокое понимание данных аспектов является необходимым условием для разработки эффективных автоматизированных средств анализа, способных адаптироваться к эволюции угроз и особенностям современных программных экосистем.

2.2 Методы и инструменты статического анализа кода (SAST)

Статический анализ исходного кода (Static Application Security Testing, SAST) представляет собой совокупность методов автоматизированной проверки программ без их выполнения, направленных на выявление дефектов безопасности, нарушений стандартов кодирования и потенциальных уязвимостей [24]. В отличие от динамического анализа (DAST), который требует запуска приложения и моделирования атак, SAST работает непосредственно с текстом программы, её абстрактными представлениями и метаданными, что позволяет обнаруживать проблемы на самых ранних этапах жизненного цикла разработки [20].

2.2.1 Принципы работы статических анализаторов

Фундаментальная задача любого SAST-инструмента — построить формальное представление программы и применить к нему набор правил или алгоритмов для выявления отклонений от ожидаемого безопасного поведения. Типичный конвейер статического анализатора включает следующие этапы:

1. **Препроцессинг и лексический анализ.** На первом шаге исходный код превращается в поток токенов: попутно обрабатываются директивы препроцессора, комментарии и макросы. В языках семейства C/C++ это само по себе нетривиальная задача — условная компиляция и макросы с побочными эффектами могут кардинально изменить смысл внешне одинаковых фрагментов [12].
2. **Синтаксический анализ (парсинг).** Из полученного потока токенов строится абстрактное синтаксическое дерево (Abstract Syntax Tree, AST), воспроизводящее иерархическую структуру программы. Современные парсеры — Clang, Tree-sitter, ANTLR — поддерживают инкрементальный режим и способны продолжать анализ даже при наличии синтаксических ошибок [2].

3. **Семантический анализ и построение промежуточных представлений.** Наряду с AST анализатор выстраивает несколько дополнительных структур:

- *Control Flow Graph (CFG)* — граф потока управления, где узлы соответствуют базовым блокам кода, а рёбра обозначают возможные переходы между ними;
- *Program Dependence Graph (PDG)* — объединяет зависимости по данным (data dependencies) и по управлению (control dependencies), тем самым позволяя видеть взаимовлияние операций сразу в двух плоскостях;
- *Data Flow Graph (DFG)* — граф, отражающий, где и как порождаются значения переменных и каким образом они распространяются между операциями в ходе выполнения программы.

4. **Применение правил анализа.** На построенных представлениях выполняются алгоритмы поиска уязвимостей: сопоставление с шаблонами, анализ потока данных, абстрактная интерпретация.

5. **Генерация отчёта.** Найденные проблемы аннотируются метаданными: тип уязвимости (CWE), уровень критичности (CVSS), локализация в коде, трассировка потока данных и рекомендации по исправлению. Современные инструменты поддерживают стандарт SARIF (Static Analysis Results Interchange Format) — формат OASIS для обмена результатами статического анализа, что обеспечивает совместимость с CI/CD-системами, IDE и платформами безопасности [5, 26].

2.2.2 Представления кода в виде графов

Для глубокого анализа семантики программы современные SAST-инструменты и методы машинного обучения используют различные типы графовых структур, каждая из которых захватывает определённые аспекты поведения кода [16, 1]:

- **AST (Abstract Syntax Tree)** — иерархическое дерево, отражающее синтаксическую структуру программы согласно грамматике языка программирования. Корень дерева соответствует всей программе, листья — токенам (идентификаторам, литералам, операторам). AST сохраняет вложенность конструкций (циклы, условия, вызовы функций), но не отражает порядок выполнения операций.
- **CFG (Control Flow Graph)** — ориентированный граф, вершины которого соответствуют базовым блокам кода (последовательностям операторов без переходов), а рёбра — возможным переходам управления (условные ветвления, циклы, вызовы функций, возвраты). CFG

позволяет анализировать достижимость кода, выявлять мёртвый код и строить пути выполнения.

- **DFG (Data Flow Graph)** — граф зависимостей по данным, где вершины представляют операции или переменные, а направленные рёбра показывают передачу значений между ними. DFG критически важен для обнаружения уязвимостей, связанных с распространением “загрязнённых” данных (tainted data flow), таких как инъекции или переполнения буфера.
- **PDG (Program Dependence Graph)** — надстройка над CFG и DFG, включающая рёбра двух типов: зависимости по управлению (control dependence) и зависимости по данным (data dependence). Благодаря этому PDG даёт наиболее полную картину программы: в одном графе оказываются видны и пути выполнения, и распространение значений. Именно поэтому он считается предпочтительной основой для продвинутых методов обнаружения уязвимостей на базе графовых нейронных сетей [17, 15].

На рисунке 2 приведён пример функции на языке C и её представления в виде трёх графов. AST раскрывает иерархическую вложенность операторов, CFG показывает возможные пути выполнения с учётом ветвлений, PDG — комплексные зависимости, объединяющие управление и данные. В настоящей работе применяется упрощённое представление на основе скользящего окна (sliding window) для формирования матрицы смежности: такой подход позволяет найти разумный баланс между выразительностью графовой структуры и вычислительными затратами [1].

2.2.3 Классификация подходов к статическому анализу

Методы SAST принято разграничивать по степени формальности, достигаемой точности и вычислительной сложности. Традиционно выделяют четыре основные категории [24, 23]:

Сигнатурный анализ и регулярные выражения Наиболее лёгкий в реализации и быстрый в работе подход: анализатор ищет в исходном коде заранее составленные шаблоны — сигнатуры известных проблем. Типичные примеры правил:

- `strcpy(\s*,\s*.)` — признак потенциального переполнения буфера;
- `system(\s.*\w+)` — возможная точка инъекции команд;
- `==\s["'"].*["'"]` — сравнение хэшей в обход криптографически стойких функций.

Преимущества метода: высокая скорость, низкие требования к ресурсам, простота добавления новых правил. Ограничения: неспособность обнаруживать сложные логические уязвимости,

зависимость от ручного обновления сигнатур, высокий уровень ложных срабатываний при неточных регулярных выражениях [13]. Инструменты, использующие данный подход: **grep**-скрипты, **Flawfinder**, **RIPS** (для PHP).

Анализ потока данных (Data Flow Analysis, Taint Analysis) Более совершенный метод, отслеживающий распространение “загрязнённых” (tainted) данных от источников ввода (user input, network, files) к критическим точкам использования (sinks), таким как выполнение команд, запросы к БД или вывод в браузер [14]. Алгоритм работает в два этапа:

1. *Marking sources*: идентификация точек входа внешних данных (например, `argv`, `HTTP_POST`, `fread`).
2. *Propagation*: распространение метки “tainted” через присваивания, арифметические операции, вызовы функций.
3. *Sink checking*: проверка, достигает ли tainted-данных критическая функция (например, `system`, `eval`, `printf` с форматной строкой).

Анализ потока данных может быть *поточным* (flow-sensitive) — учитывающим порядок выполнения операторов, или *непоточным* (flow-insensitive) — более быстрым, но менее точным. Современные реализации (CodeQL, Semgrep Pro) используют межпроцедурный анализ (interprocedural), отслеживая tainted-данные через границы функций [21].

Абстрактная интерпретация Формальный метод, предложенный П. и Р. Кузо в 1977 году, основанный на аппроксимации множества всех возможных состояний программы [25]. Вместо выполнения программы на конкретных входных данных, абстрактная интерпретация работает с *абстрактными доменами* (интервалы, множества, предикаты), что позволяет доказывать отсутствие определённых классов ошибок для *всех* возможных входов.

Пример: для проверки переполнения буфера анализатор может использовать домен интервалов $[min, max]$ для переменных-индексов. Если при обращении к массиву $arr[i]$ доказано, что $i \notin [0, len(arr) - 1]$ для любого возможного выполнения, то уязвимость отсутствует.

Преимущества: высокая точность, возможность формальной верификации. Ограничения: вычислительная сложность, необходимость ручного указания инвариантов, проблемы с масштабированием на большие кодовые базы [2]. Инструменты: **Astrée** (для встраиваемых систем), **Frama-C** (для C), **Infer** (Facebook/Meta).

2.2.4 Обзор популярных решений

На рынке статического анализа представлено множество инструментов, различающихся по лицензии, поддерживаемым языкам, точности и интеграционным возможностям. Условно их можно разделить на коммерческие и открытые [24].

Коммерческие решения

- **SonarQube** (SonarSource) — платформа для непрерывной инспекции кода, поддерживающая 30+ языков. Использует комбинацию правил на основе AST, анализа потока данных и машинного обучения для ранжирования проблем. Интегрируется с CI/CD (Jenkins, GitLab CI, GitHub Actions). Поддерживает кастомные правила на Java/Python и экспорт отчётов в формате SARIF [26].
- **Checkmarx** — enterprise-решение с акцентом на безопасность. Предоставляет межпроцедурный tainted-анализ, обнаружение уязвимостей OWASP Top 10, генерацию отчётов для аудитов. Поддерживает SCA (Software Composition Analysis) для выявления уязвимых зависимостей [27].
- **Fortify Static Code Analyzer** (OpenText) — один из старейших SAST-инструментов. Использует семантический анализ на основе CFG/PDG, поддерживает более 25 языков. Отличается низкой частотой ложных срабатываний благодаря контекстному анализу [28].
- **CodeQL** (GitHub) — язык запросов для семантического анализа кода. Позволяет писать рекурсивные запросы для поиска сложных паттернов (например, “найти все пути от user input до SQL-запроса”). Интегрирован в GitHub Advanced Security, поддерживает автодообучение на новых уязвимостях [29].

Open-source решения

- **Semgrep** — современный анализатор с декларативным языком правил. Поддерживает как сигнатурный поиск, так и анализ потока данных через taint-mode. Быстрый (написан на OCaml), легко интегрируется в pre-commit хуки [30].
- **Bandit** — инструмент для анализа кода на Python. Использует AST-парсинг и набор правил для выявления распространённых уязвимостей (hardcoded passwords, unsafe YAML, SQL injection). Расширяем через плагины [31].
- **Flawfinder** — простой сигнатурный сканер для C/C++. Ищет опасные функции (strcpy, gets, system) и оценивает риск по шкале 0–5. Не требует компиляции, но даёт много ложных срабатываний [32].

- **Cppcheck** — анализатор для C/C++ с акцентом на обнаружение ошибок времени выполнения (out-of-bounds, null pointer, memory leaks). Использует интерпроцедурный анализ потока данных, но не поддерживает межфайловую проверку [33].

2.2.5 Ограничения традиционных подходов

Несмотря на зрелость технологий SAST, существующие инструменты сталкиваются с рядом фундаментальных ограничений, особенно в контексте современных практик разработки [2, 3]:

1. **Высокий уровень ложных срабатываний (False Positives).** Сигнатурные методы часто помечают безопасный код как уязвимый, если он содержит потенциально опасные функции без учёта контекста. Например, вызов `strcpy` в коде, где длина источника гарантированно меньше размера буфера, будет помечен как уязвимость, хотя на самом деле безопасен. Это порождает эффект «усталости от предупреждений» (alert fatigue): разработчики начинают игнорировать отчёты анализатора целиком [20].
2. **Пропуск сложных логических уязвимостей (False Negatives).** Традиционные анализаторы плохо справляются с дефектами, для выявления которых недостаточно синтаксического разбора — нужно понимать семантику программы. Race conditions, логические ошибки уровня бизнес-логики, уязвимости, проявляющиеся лишь при определённой последовательности вызовов, — всё это нередко остаётся незамеченным вплоть до момента реальной эксплуатации [21].
3. **Зависимость от ручного обновления правил.** Сигнатурные системы живут ровно до тех пор, пока кто-то регулярно добавляет в них новые шаблоны. При нынешней скорости появления свежих классов атак — взять хотя бы уязвимости в LLM-интеграциях — этот процесс превращается в «гонку вооружений», в которой защищающаяся сторона по определению запаздывает [23].
4. **Проблемы масштабирования.** Межпроцедурный анализ потока данных и символическое выполнение имеют высокую вычислительную сложность. Для больших кодовых баз (миллионы строк) время анализа может исчисляться часами, что делает невозможным интеграцию в быстрые CI/CD-конвейеры [15].
5. **Недостаточная интерпретируемость.** Многие инструменты выдают отчёты в формате «строка X: потенциальная уязвимость типа Y», без объяснения *почему* код считается уязвимым и *как* его исправить. Это затрудняет работу разработчиков, особенно с низким уровнем экспертизы в безопасности [5].

6. **Языковая и фреймворк-зависимость.** Большинство анализаторов оптимизированы под конкретные языки (C/C++, Java, Python) и не поддерживают современные фреймворки (React, Django, Spring Boot) или DSL (domain-specific languages). Это создаёт “слепые зоны” в гетерогенных проектах [3].

Указанные ограничения мотивируют поиск новых подходов к анализу уязвимостей, в частности — применение методов машинного обучения и больших языковых моделей, способных извлекать семантические представления кода, обобщать паттерны из данных и генерировать объяснимые рекомендации [20, 21]. В рамках данной работы предлагается гибридная архитектура, сочетающая графовые нейронные сети (ReGVD) для структурного анализа, LoRA-адаптеры для эффективного дообучения и FAISS для мгновенной реакции на новые уязвимости, что позволяет частично преодолеть перечисленные недостатки традиционных SAST-инструментов [1].

2.3 Применение методов машинного обучения в задачах кибербезопасности

Переход от эвристических и сигнатурных методов к алгоритмам машинного обучения (ML) ознаменовал принципиальный сдвиг в парадигме статического анализа кода. Вместо жёстких правил, требующих ручного сопровождения, современные подходы опираются на извлечение закономерностей из размеченных датасетов, что позволяет автоматизировать обнаружение уязвимостей, классификацию их типов и генерацию рекомендаций по исправлению [20]. В зависимости от способа представления кода и архитектуры моделей, методы ML для анализа уязвимостей принято разделять на три крупные группы: классическое машинное обучение на инженерных признаках, глубокое обучение с графовыми представлениями (GNN) и большие языковые модели (LLM).

2.3.1 Классическое машинное обучение для анализа кода

Классические ML-алгоритмы (Random Forest, Gradient Boosting, Support Vector Machines, логистическая регрессия) требуют предварительного преобразования исходного кода в структурированные числовые векторы (feature engineering). Типичный конвейер извлечения признаков включает следующие этапы [3, 2]:

- **Метрики абстрактного синтаксического дерева (AST):** глубина дерева, количество узлов каждого типа, распределение операторов и выражений, максимальная вложенность циклов/условий.
- **Метрики графа потока управления (CFG):** цикломатическая сложность (McCabe),

число базовых блоков, количество путей выполнения, плотность рёбер, наличие недостижимого кода.

- **Метрики графа зависимостей (PDG):** длина цепочек зависимостей по данным, плотность связей между переменными, число точек синхронизации, степень связности компонентов.
- **Статистические и лексические признаки:** частота n-грамм токенов, распределение имён переменных, наличие “магических чисел”, соотношение комментариев к коду, размер функции в строках.
- **API-последовательности и вызовы библиотек:** частота обращения к опасным функциям (`strcpy`, `malloc`, `exec`), порядок вызовов системных API, использование паттернов обработки ошибок.

После векторизации применяются алгоритмы классификации. Наиболее устойчивые результаты демонстрируют ансамблевые методы: Random Forest обеспечивает высокую интерпретируемость через feature importance, а XGBoost/LightGBM эффективно работают с несбалансированными данными благодаря взвешенным функциям потерь [3].

Преимущества классического подхода:

1. **Высокая скорость инференса:** предсказание занимает микросекунды, что позволяет интегрировать анализ в pre-commit хуки и быстрые CI/CD-конвейеры.
2. **Низкие вычислительные требования:** модели работают на CPU, не требуют GPU-кластеров и могут разворачиваться на edge-устройствах.
3. **Интерпретируемость:** можно отследить, какие признаки повлияли на решение (например, “высокая цикломатическая сложность + вызов `system` = риск инъекции”).

Ограничения:

1. **Зависимость от качества признаков:** ручная инженерия метрик трудоёмка и не охватывает семантические нюансы кода.
2. **Потеря контекста и длинных зависимостей:** плоские векторы не сохраняют информацию о взаимном расположении конструкций в коде.
3. **Чувствительность к рефакторингу:** переименование переменных или изменение порядка операторов может радикально изменить вектор признаков, хотя семантика остаётся прежней.

4. **Потребность в полном переобучении при появлении новых данных:** классические ML-модели не поддерживают инкрементальное обновление. При поступлении новых классов уязвимостей или изменении распределения данных требуется повторное извлечение признаков и обучение на объединённом датасете, что существенно увеличивает время реакции на новые угрозы и вычислительные затраты [15].

2.3.2 Глубокое обучение и графовые нейронные сети (GNN)

Преодоление ограничений feature-based методов связано с переходом к автоматическому обучению представлений (representation learning). Исходный код естественно моделируется как граф, где вершины соответствуют токенам, операторам или базовым блокам, а рёбра отражают синтаксические, потоковые или семантические связи [16, 1].

Devign: базовая архитектура и ограничения Первой прорывной работой в этой области стала модель Devign [16], которая комбинирует CFG и PDG, применяет gated GNN-слои и max-pooling для агрегации графа в вектор. Модель обучалась на датасете BigVul, содержащем около 27 000 функций на языках C/C++ из открытых репозиториях GitHub. Разметка выполнялась вручную экспертами по безопасности на основе анализа CVE-идентификаторов и коммитов, исправляющих уязвимости (“fixing commits”). Несмотря на заявленный F1-score ≈ 0.62 , оригинальная реализация Devign не была опубликована в открытом доступе, что делает невозможной независимую верификацию результатов, воспроизведение метрик и адаптацию под современные фреймворки. Отсутствие открытого кода и воспроизводимых бенчмарков стало существенным барьером для дальнейшего развития направления.

ReGVD: эволюция подхода и открытость Следующим этапом стала модель ReGVD [1], предлагающая архитектурные и методологические улучшения, повысившие F1-score до ≈ 0.72 на том же датасете BigVul:

1. *Учёт последовательности токенов:* помимо структурных графов, модель использует скользящее окно (sliding window) для построения матрицы смежности, отражающей линейный порядок кода. Это компенсирует потерю синтаксического контекста при переходе от AST к CFG/PDG.
2. *Residual connections и soft attention pooling:* первый механизм предотвращает затухание градиентов в глубоких GNN-слоях, второй позволяет модели самостоятельно расставлять акценты на значимых фрагментах функции перед финальной классификацией.

Отдельного внимания заслуживает то, что ReGVD распространяется с полностью открытым исходным кодом и воспроизводимыми скриптами обучения — это обеспечивает прозрачность заявленных результатов и возможность независимой проверки. В более широкой перспективе ReGVD можно рассматривать как своеобразную точку зрелости классических графовых нейронных сетей в задаче детекции уязвимостей: начиная примерно с 2023 года как индустрия, так и академическое сообщество заметно переориентировались на большие языковые модели, привлекающие возможностями zero-shot обобщения и автоматической генерации исправлений. Вместе с тем GNN-подходы по-прежнему востребованы там, где принципиальны детерминированная работа с графами зависимостей и минимальная латентность инференса.

2.3.3 Большие языковые модели (LLM) в анализе уязвимостей

Появление архитектуры Transformer и моделей, заточенных под работу с кодом, — StarCoder, CodeLlama, DeepSeek-Coder, GPT-4o — сформировало принципиально новый класс методов анализа, опирающихся на семантическое понимание как программного, так и естественного языка [20, 23].

Преимущества и механизмы применения LLM умеют работать в режимах zero-shot и few-shot, строить цепочки рассуждений (chain-of-thought reasoning) для объяснения природы уязвимости, а также предлагать патчи и содержательные комментарии. Немаловажно, что одна и та же архитектура без каких-либо изменений справляется с кодом на десятках языков программирования [21].

Фундаментальные ограничения Высокие результаты в бенчмарках, однако, не гарантируют беспрепятственного использования в production. На практике интеграция LLM в промышленные сканеры наталкивается на ряд серьёзных препятствий [2, 23]:

- **Проблема обучающих данных:** модели натренированы на статических датасетах — CodeSearchNet, The Pile и подобных — которые быстро теряют актуальность и не охватывают свежие паттерны уязвимостей или недавно появившиеся фреймворки.
- **Сложность и стоимость дообучения:** полная настройка моделей от 7B параметров и выше требует сотен гигабайт GPU-памяти и обширных размеченных датасетов. Параметрозэффективные методы — LoRA, QLoRA — снижают издержки, однако не решают проблему по существу.
- **Зависимость от промпт-инжиниринга:** точность детекции во многом определяется формулировкой промпта, системными инструкциями и подбором примеров в контексте. Для

каждого класса уязвимостей всё это приходится настраивать и тестировать вручную.

- **Неструктурированный вывод:** LLM возвращают текстовые описания, которые трудно автоматически разобрать для встраивания в CI/CD-пайплайн. Отсутствие единого формата отчётности существенно затрудняет триаж, приоритизацию и автоматическое закрытие тикетов.
- **Галлюцинации и отсутствие формальных гарантий:** вероятностная природа генерации исключает возможность доказать отсутствие уязвимостей, а «выдуманные» патчи рискуют внести в код новые дефекты безопасности.

2.3.4 Сравнительный анализ и обоснование гибридного подхода

Проведённый анализ архитектурных особенностей и экспериментальных результатов рассмотренных методов показывает, что ни один из подходов не способен в полной мере решить задачу обнаружения уязвимостей в условиях постоянно изменяющегося ландшафта угроз. Классические алгоритмы машинного обучения отличаются высокой скоростью работы и относительной интерпретируемостью результатов, однако их эффективность существенно зависит от качества ручной инженерии признаков. Кроме того, такие модели плохо адаптируются к появлению новых типов уязвимостей и требуют регулярного переобучения.

Графовые нейронные сети, в частности архитектура ReGVD, позволяют автоматически извлекать структурные и семантические представления исходного кода, а также сохраняют устойчивость к изменениям, связанным с рефакторингом. Вместе с тем подобные модели характеризуются высокой вычислительной стоимостью и подвержены эффекту *catastrophic forgetting* при обновлении данных, что осложняет их применение в динамической среде.

Большие языковые модели демонстрируют высокую гибкость при анализе программного кода, в том числе в сценариях zero-shot и few-shot обучения. Однако их использование в промышленных системах ограничивается значительными затратами вычислительных ресурсов, высокой латентностью, зависимостью от качества промптов и сложностью автоматизированной обработки результатов для последующего триажа.

Сводное сравнение рассмотренных подходов, основанное на результатах оригинальных исследований, приведено в таблице 9.

Выявленные ограничения обусловили необходимость разработки **гибридной архитектуры**, в которой преимущества различных методов используются совместно, а их недостатки компенсируются за счёт взаимодействия компонентов системы. В рамках данной работы реализован многоурневый конвейер, основанный на следующих принципах.

1. **Разделение функций между компонентами и объединение оценок.** Графовая модель ReGVD выполняет структурный анализ кода и формирует вероятность наличия уязвимости P_{cls} . Дополнительно используется модуль семантического поиска на базе FAISS, который позволяет учитывать схожие ранее обнаруженные примеры и вычисляет значение S_{faiss} — долю уязвимых объектов среди $K = 5$ ближайших соседей в пространстве эмбедингов. Итоговый скор определяется выражением:

$$S_{hybrid} = \beta \cdot P_{cls} + (1 - \beta) \cdot S_{faiss},$$

где $\beta \in [0, 1]$ подбирается на валидационной выборке методом *grid search*. В базовой конфигурации используется значение $\beta = 0.6$, обеспечивающее баланс между способностью GNN к обобщению и адаптивностью поискового компонента.

2. **Параметро-эффективная адаптация модели.** Для уменьшения вычислительных затрат основная часть параметров GraphCodeBERT остаётся замороженной, а адаптация выполняется с помощью LoRA-модулей ($r = 8$, $\alpha = 16$), внедрённых в проекции query/value. Благодаря этому количество обучаемых параметров составляет менее одного процента от общего объёма модели. При накоплении новых данных выполняется периодическое дообучение с уменьшенной скоростью обучения ($1 \cdot 10^{-4}$) и использованием replay-буфера с коэффициентом 0.2, что позволяет снизить риск потери ранее усвоенных знаний.
3. **Автоматизированный триаж результатов.** Значение гибридного скор S_{hybrid} используется для ранжирования обнаруженных потенциальных уязвимостей. Если $S_{hybrid} < 0.3$, фрагмент кода считается безопасным; диапазон $0.3 \leq S_{hybrid} < 0.6$ соответствует случаям, требующим дополнительной ручной проверки; при $S_{hybrid} \geq 0.6$ уязвимость классифицируется как критическая. Подобная схема позволяет существенно сократить количество ложных срабатываний и уменьшить нагрузку на специалистов по информационной безопасности.
4. **Оперативное обновление базы знаний.** При появлении новых или ранее неизвестных уязвимостей система может быть обновлена без остановки основного конвейера. Для этого достаточно добавить соответствующий эмбединг в FAISS-индекс с помощью вызова `index.add()`. Такая операция выполняется за короткое время, не требует повторного полного обучения модели и обеспечивает быструю адаптацию системы к новым угрозам.

Предложенная архитектура объединяет преимущества нескольких подходов: графовая модель обеспечивает устойчивость и способность к обобщению, FAISS-компонент отвечает за адаптацию к новым данным, а механизм взвешенного комбинирования позволяет поддерживать ба-

ланс между стабильностью и гибкостью системы. Разработанное решение ориентировано на интеграцию в CI/CD-инфраструктуру, поддерживает экспорт результатов в формате SARIF и обеспечивает воспроизводимый процесс анализа исходного кода, что будет подтверждено в экспериментальной части работы.

3 Разработка гибридной архитектуры сканера уязвимостей на основе ReGVD, LoRA и FAISS

3.1 Базовая архитектура: модель ReGVD

Фундаментальным компонентом разрабатываемой гибридной системы выступает архитектура ReGVD (Revisiting Graph Neural Networks for Vulnerability Detection), первоначально представленная в работе Nguyen et al. на конференции ICSE 2022 [1]. Данная модель представляет собой эволюционное развитие подхода Devign [16] и решает ряд фундаментальных ограничений предыдущих графовых нейронных сетей, применяемых для статического анализа кода. В контексте данной курсовой работы ReGVD модифицирована и расширена дополнительными модулями, однако базовый конвейер обработки и механизмы агрегации признаков сохранены в соответствии с оригинальной спецификацией.

Архитектура ReGVD строится на многоэтапном пайплайне преобразования исходного кода функции в векторное представление, пригодное для бинарной классификации. Первым этапом является семантическое кодирование токенов с использованием предобученной языковой модели GraphCodeBERT [17]. В рамках данной реализации веса GraphCodeBERT (общее количество параметров ≈ 125 млн) замораживаются на этапе начального обучения, а модель выступает в роли фиксированного encoder'a, преобразующего линейную последовательность токенов функции в плотные контекстуальные эмбединги размерности 768. Выбор GraphCodeBERT обусловлен его архитектурой, которая на этапе pre-training учитывает не только синтаксическую последовательность, но и графы потока данных (data flow edges), что обеспечивает более точное начальное представление семантики программы по сравнению с классическими CodeBERT или RoBERTa [2, 23].

На следующем этапе линейная последовательность токенов преобразуется в структурное представление кода — ориентированный граф. В отличие от подходов, требующих полного парсинга AST или CFG (что вычислительно затратно, привязано к конкретному языку и чувствительно к синтаксическим ошибкам), ReGVD использует эвристический метод построения графа на основе скользящего окна (sliding window). Для каждого токена устанавливаются направленные рёбра к соседним токенам в пределах окна размером w (по умолчанию $w = 5$). Матрица смежности $A \in \{0, 1\}^{N \times N}$ формируется следующим образом:

$$A_{ij} = \begin{cases} 1, & \text{если } |i - j| \leq w \text{ и } i \neq j \\ 0, & \text{в противном случае} \end{cases}$$

где N — количество токенов в анализируемой функции. Подход захватывает локальные синтаксические и контекстуальные зависимости, сохраняя при этом линейную сложность построения

графа $O(N \cdot w)$, что принципиально важно при обработке объёмных функций и обеспечивает масштабируемость на крупных репозиториях [1, 3].

Сформированный граф с узловыми эмбедингами $\mathbf{H}^{(0)} \in \mathbb{R}^{N \times 768}$ поступает на вход графовой нейронной сети. В работе реализованы два варианта архитектурных блоков: ReGCN (на базе Graph Convolutional Networks) и ReGGNN (на базе Gated Graph Neural Networks). Отличительной чертой ReGVD служат residual connections (остаточные связи) на каждом GNN-слое: они предотвращают oversmoothing и затухание градиентов при наращивании глубины сети. Обновление представлений узлов на слое l описывается формулой:

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right) + \mathbf{H}^{(l)},$$

где $\tilde{A} = A + I$ — матрица смежности с добавленными самосвязями, $\tilde{D}$ — соответствующая диагональная матрица степеней, $\mathbf{W}^{(l)}$ — обучаемые веса линейного преобразования, σ — функция активации (GELU), а слагаемое $\mathbf{H}^{(l)}$ обеспечивает прямую передачу исходных признаков и стабилизирует обучение [1, 15].

После прохождения L GNN-слоёв (в данной реализации $L = 2$) узловые представления агрегируются в единый вектор всей функции. Вместо стандартного max/mean pooling в ReGVD применяется механизм soft attention pooling. Для каждого узла вычисляется нормализованный вес внимания:

$$\alpha_i = \frac{\exp \left(\mathbf{u}^\top \tanh \left(\mathbf{W}_p \mathbf{h}_i^{(L)} \right) \right)}{\sum_{j=1}^N \exp \left(\mathbf{u}^\top \tanh \left(\mathbf{W}_p \mathbf{h}_j^{(L)} \right) \right)},$$

где $\mathbf{W}_p \in \mathbb{R}^{d \times d}$ и $\mathbf{u} \in \mathbb{R}^d$ — обучаемые параметры механизма внимания. Итоговое представление графа вычисляется как взвешенная сумма:

$$\mathbf{h}_{graph} = \sum_{i=1}^N \alpha_i \mathbf{h}_i^{(L)}.$$

Благодаря этому механизму модель самостоятельно выделяет семантически значимые фрагменты — вызовы потенциально опасных функций, точки ввода/вывода, операции с памятью, — отодвигая на второй план шумовые и тривиальные конструкции. На несбалансированных датасетах это заметно повышает полноту обнаружения [1, 21].

Рис. 1: Архитектура гибридного сканера на базе ReGVD: конвейер обработки исходного кода от токенизации до формирования итогового скор. Пунктирными рамками выделены компоненты, интегрированные в рамках данной работы (LoRA-адаптеры, Embedding Head, FAISS-индекс, гибридный скор).

Полученное представление $\mathbf{h}_{graph}$ далее поступает в классификационный модуль, реализованный в виде однослойного персептрона с функцией активации Sigmoid. На выходе формируется

вероятность наличия уязвимости $P_{cls} \in [0, 1]$.

Эффективность архитектуры ReGVD объясняется тем, что модель одновременно учитывает как локальные особенности кода, извлекаемые с помощью механизма sliding window, так и глобальные зависимости между элементами программы, моделируемые графовой нейронной сетью. Согласно результатам, опубликованным для датасетов BigVul и ReVeal, такой подход обеспечивает прирост F1-score примерно на 5–12% по сравнению с моделью Devign [16, 1].

Несмотря на высокое качество детекции, базовая версия ReGVD наследует ряд ограничений, характерных для статических GNN-архитектур. В частности, при появлении новых типов уязвимостей требуется повторное обучение модели, отсутствует механизм быстрой адаптации к zero-day сценариям, а сформированное признаковое пространство остаётся фиксированным после завершения обучения. Для устранения этих недостатков в предложенной системе архитектура была дополнена embedding-модулем для семантического поиска, LoRA-адаптерами для параметро-эффективного дообучения и гибридным механизмом принятия решений, которые рассматриваются далее.

3.2 Модуль семантического поиска: Embedding Head + FAISS

Для повышения адаптивности системы к новым типам уязвимостей в архитектуру был интегрирован модуль семантического поиска, объединяющий embedding-голову и векторную базу данных на основе FAISS. Такой подход позволяет реализовать механизм «горячего обновления» (*hot update*), при котором система способна учитывать новые паттерны кода без переобучения основной нейросетевой модели [1, 18].

Embedding-голова представляет собой двухслойный нелинейный проектор. На вход она получает агрегированное графовое представление функции $\mathbf{h}_{graph} \in \mathbb{R}^d$, сформированное GNN-модулем, после чего отображает его в компактное семантическое пространство размерности $E = 512$. Преобразование описывается следующим выражением:

$$\mathbf{z} = \frac{\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \mathbf{h}_{graph} + \mathbf{b}_1) + \mathbf{b}_2}{\|\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \mathbf{h}_{graph} + \mathbf{b}_1) + \mathbf{b}_2\|_2},$$

где $\mathbf{W}_1 \in \mathbb{R}^{h \times d}$ и $\mathbf{W}_2 \in \mathbb{R}^{E \times h}$ являются обучаемыми матрицами весов, а h соответствует размерности скрытого слоя.

Использование L_2 -нормализации обеспечивает приведение всех итоговых эмбеддингов $\mathbf{z}$ к единичной гиперсфере $\mathbb{S}^{E-1}$. Благодаря этому операция скалярного произведения становится эквивалентной косинусному сходству, что упрощает вычисление близости между векторами и повышает устойчивость алгоритмов поиска ближайших соседей (Approximate Nearest Neighbor, ANN) [19].

Обучение embedding-проектора выполняется совместно с классификационной частью модели в рамках многозадачной оптимизации. На первом этапе обучения (Фаза 1) используется комбинированная функция потерь, включающая бинарную кросс-энтропию и Supervised Contrastive Loss (SupCon):

$$\mathcal{L}_{total} = \alpha \cdot \mathcal{L}_{BCE} + (1 - \alpha) \cdot \mathcal{L}_{SupCon},$$

где гиперпараметр $\alpha \in [0, 1]$ регулирует баланс между задачами. В данной реализации принято $\alpha = 0.7$: такое значение отдаёт приоритет прямой классификации, сохраняя при этом достаточный градиентный сигнал для кластеризации эмбедингов [1]. Для мини-батча размера N формула Supervised Contrastive Loss записывается как:

$$\mathcal{L}_{SupCon} = -\frac{1}{N} \sum_{i=1}^N \frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\mathbf{z}_i^\top \mathbf{z}_p / \tau)}{\sum_{a \in A(i)} \exp(\mathbf{z}_i^\top \mathbf{z}_a / \tau)},$$

где $P(i)$ — множество индексов семплов того же класса, что и i -й (positive pairs), $A(i)$ — все прочие семплы батча, $\tau = 0.07$ — температурный параметр, управляющий «остротой» распределения сходств. В отличие от классического Triplet Loss, SupCon сравнивает примеры сразу по всему батчу, что обеспечивает более устойчивые градиенты и лучшее разделение кластеров в пространстве эмбедингов при том же объёме батча [19]. Дополнительно задействована стратегия online hard mining: оптимизация автоматически акцентируется на наиболее трудноразличимых парах — уязвимых снippets, семантически близких к безопасным, и наоборот.

После завершения начального этапа обучения все объекты обучающей выборки пропускаются через модель для получения эмбедингов $\{\mathbf{z}_1, \dots, \mathbf{z}_M\}$. Для каждого вектора дополнительно сохраняются сопутствующие данные: бинарная метка уязвимости, идентификатор примера в датасете и соответствующий фрагмент исходного кода. Полученные представления загружаются в векторную базу данных, построенную на основе FAISS (Facebook AI Similarity Search) [18].

Выбор FAISS обусловлен тем, что библиотека обеспечивает высокую скорость поиска в плотных векторных пространствах и хорошо подходит для задач анализа исходного кода. В рамках разработанной системы ключевыми являются следующие характеристики.

1. **Высокая производительность поиска.** Поиск ближайших соседей в индексах размером 10^5 – 10^6 векторов выполняется за миллисекунды даже на CPU, что позволяет использовать систему в автоматизированных CI/CD-конвейерах без заметного увеличения времени анализа.
2. **Поддержка инкрементального обновления.** Метод `index.add()` позволяет добавлять новые эмбединги без полного пересоздания индекса. Благодаря этому система может обновляться в рабочем режиме без остановки сервиса.

3. Масштабируемость индексов. Для сравнительно небольших наборов данных (до 10^5 записей) используется индекс `IndexFlatIP`, реализующий точный поиск по внутреннему произведению. При увеличении объёма базы возможен переход к `IndexIVFFlat`, который обеспечивает существенное ускорение поиска при минимальной потере точности [18, 23].

Во время инференса (Фаза 2) анализируемый фрагмент кода преобразуется в эмбединг $\mathbf{z}_{query}$, который передаётся в FAISS для поиска K ближайших соседей (в базовой конфигурации используется $K = 5$). В результате система получает индексы наиболее похожих векторов и расстояния до них. На основе меток найденных соседей вычисляется значение FAISS-скора:

$$S_{faiss} = \frac{1}{K} \sum_{j=1}^K \mathbb{I}(y_j = 1),$$

где $y_j \in \{0, 1\}$ обозначает метку уязвимости j -го ближайшего соседа, а $\mathbb{I}(\cdot)$ является индикаторной функцией. Таким образом, S_{faiss} показывает долю уязвимых примеров в локальной семантической окрестности исследуемого фрагмента и принимает значения в диапазоне $[0, 1]$.

Механизм горячего обновления активируется при поступлении новой размеченной выборки. В этом случае новые примеры проходят через уже обученную и замороженную базовую модель: параметры `GraphCodeBERT`, графовых слоёв и основных проекторов не изменяются. Для новых объектов вычисляются эмбединги, после чего они добавляются в индекс с помощью вызова `faiss_index.add()`. Такая процедура занимает минимальное время и не требует повторного полного обучения модели, что позволяет оперативно учитывать zero-day уязвимости и доменно-специфичные паттерны кода [2, 21].

Совместное использование embedding-модуля и FAISS значительно повышает адаптивность системы. Графовый классификатор отвечает за выявление структурных закономерностей в коде и обобщение знаний, тогда как FAISS реализует поиск по семантическому сходству и позволяет быстро находить похожие примеры в базе данных. Особенно полезен такой подход при анализе модифицированных версий уже известных уязвимостей, например при переименовании функций, обфускации кода или изменении реализации криптографических примитивов. Несмотря на внешние различия, подобные фрагменты обычно сохраняют близость в пространстве эмбедингов, что делает их обнаружение более устойчивым по сравнению с классическими сигнатурными методами [20, 3].

В то же время поиск по аналогии имеет и определённые ограничения. Если в векторной базе отсутствуют семантически близкие примеры, значение S_{faiss} начинает зависеть от общего распределения меток в датасете и теряет информативность. По этой причине FAISS-скор не используется как самостоятельный критерий принятия решения, а комбинируется с вероятностью, полученной от графового классификатора, в рамках гибридной схемы детекции, описанной в

следующем разделе.

3.3 Адаптивное дообучение: внедрение LoRA-адаптеров

Одним из ключевых ограничений современных графовых нейронных сетей для анализа кода является проблема *catastrophic forgetting* — тенденция модели терять знания о ранее изученных паттернах при обучении на новых данных. В контексте детекции уязвимостей это означает, что при появлении новых классов дефектов (например, специфичных для свежего фреймворка или языка) традиционный fine-tuning требует либо полного переобучения на объединённом датасете (что вычислительно дорого), либо приводит к деградации качества на старых типах уязвимостей. Для решения данной проблемы в архитектуру разрабатываемого сканера интегрирован механизм параметро-эффективного адаптивного дообучения на основе технологии Low-Rank Adaptation (LoRA) [34].

LoRA представляет собой метод адаптации больших предобученных моделей, основанный на гипотезе о том, что изменения весов в процессе дообучения имеют низкий эффективный ранг. Вместо обновления всех параметров матрицы $\mathbf{W} \in \mathbb{R}^{d \times k}$, LoRA замораживает исходные веса и добавляет к ним обучаемое низкоранговое приращение:

$$\mathbf{W}' = \mathbf{W} + \Delta\mathbf{W} = \mathbf{W} + \mathbf{B}\mathbf{A},$$

где $\mathbf{A} \in \mathbb{R}^{r \times k}$ и $\mathbf{B} \in \mathbb{R}^{d \times r}$ — обучаемые матрицы, а гиперпараметр $r \ll \min(d, k)$ определяет ранг адаптера. В рамках данной реализации используется $r = 8$, что соответствует стандартному диапазону 4–16, рекомендованному в оригинальной работе [34]. При прямом проходе (forward pass) входной вектор $\mathbf{x}$ преобразуется следующим образом:

$$\mathbf{h} = \mathbf{W}\mathbf{x} + \mathbf{B}(\mathbf{A}\mathbf{x}) \cdot \frac{\alpha}{r},$$

где $\alpha = 16$ — масштабный коэффициент, обеспечивающий стабильность градиентов. Инициализация матриц производится по схеме: $\mathbf{A}$ — распределение Kaiming uniform, $\mathbf{B}$ — нулевая матрица, что гарантирует, что в начале обучения адаптер ведёт себя как тождественное преобразование и не вносит искажений в предобученные представления [2].

В контексте архитектуры ReGVD LoRA-адаптеры применяются избирательно к проекционным слоям query и value в механизме multi-head attention предобученной модели GraphCodeBERT. Выбор именно этих слоёв обусловлен тем, что они непосредственно отвечают за формирование контекстных зависимостей между токенами кода, что критично для обнаружения уязвимостей, связанных с потоком данных [17]. При этом основные 125 миллионов параметров GraphCodeBERT замораживаются. В рамках разработанной системы поддерживаются два режима адаптации, отличающихся объёмом обучаемых параметров:

- **Полное обучение гибридной модели** (используется для начальной тренировки на Devign): LoRA-адаптеры на проекциях query и value во всех 12 attention-слоях GraphCodeBERT (≈ 295 тыс. параметров), GNN-слои ReGCN (≈ 300 тыс. параметров), classification head и embedding head (≈ 5 тыс. параметров суммарно). Итого ≈ 600 тыс. обучаемых параметров.
- **Параметро-эффективная адаптация к новому домену** (используется при LoRA Fine-tune на MegaVul): LoRA-адаптеры применяются только к верхним 6 attention-слоям GraphCodeBERT (≈ 147 тыс. параметров) и classification head (≈ 5 тыс. параметров). GNN-слои и embedding head замораживаются и переиспользуются в виде, полученном на Devign. Итого ≈ 152 тыс. обучаемых параметров.

Применение тонкой настройки именно к финальным сегментам нейросети при переходе на новый домен обусловлено фундаментальным свойством трансформеров: начальные блоки аккумулируют базовые синтаксические и лексические закономерности, в то время как глубокие слои концентрируются на семантике, специфичной для целевого процесса [17, 34]. Использование данного подхода позволяет сократить объем модифицируемых весов до ≈ 12.7 млн параметров. Это на два порядка меньше объема полного цикла fine-tuning (≈ 125.7 млн), что открывает возможность эксплуатации стандартных графических ускорителей с объемом памяти от 8 до 16 ГБ для задач дообучения [20].

Система спроектирована для параллельного хранения и независимого управления набором LoRA-адаптеров, оптимизированных под конкретные языки программирования или уникальные программные среды. Параметры ранга r и коэффициента масштабирования α подбираются индивидуально для каждой надстройки, что обеспечивает компромисс между емкостью модели и нагрузкой на вычислитель. В частности, для системного ПО на языках C/C++ эффективны значения $r = 8, \alpha = 16$, в то время как для языков с многоуровневой типизацией эти показатели могут быть увеличены до $r = 16, \alpha = 32$. Смена активного адаптера происходит в рамках текущего вычислительного графа без дублирования ядра GraphCodeBERT, что существенно экономит ресурсы видеопамати и упрощает работу в распределенных инфраструктурах.

Процедура адаптивного обучения разделена на два режима, отвечающих различным эксплуатационным задачам:

Режим А: Горячее обновление (Hot Update) Этот сценарий задействуется при поступлении ограниченного объема новых данных (десятки или сотни кейсов), когда требуется мгновенная коррекция поведения без изменения внутренних весов модели. Технологический стек:

1. Входящие фрагменты кода обрабатываются статической базовой моделью (GraphCodeBERT + GNN + heads) в режиме предсказания.

2. На выходе формируются 512-мерные дескрипторы, проходящие процедуру L_2 -нормализации.
3. Векторные представления интегрируются в существующий индекс FAISS методом `index.add()`, что требует времени $O(1)$ относительно объема базы.
4. Прямое обновление параметров нейросети в данном цикле не производится.

Операция занимает считанные секунды и не несет риска деградации качества на ранее изученных паттернах. Данный подход незаменим при выявлении zero-day уязвимостей, где скорость реагирования является приоритетной [21].

Режим Б: Циклическое дообучение (LoRA Fine-tune) Этот сценарий запускается, когда накопилось достаточно новых размеченных данных (от 500 штук) и появилась потребность в серьёзной подстройке внутренних признаков модели. Процедура выглядит так:

1. Готовим тренировочный набор, смешивая свежие данные с репрезентативной подборкой из архива (replay buffer). Сколько старых примеров брать — регулирует коэффициент `replay_ratio` (по умолчанию 0.2).
2. Настраиваем только специфичные компоненты: модули LoRA, графовые слои и выходные классификаторы. Целевая функция — комбинация $\mathcal{L}_{total} = \alpha \cdot \mathcal{L}_{BCE} + (1 - \alpha) \cdot \mathcal{L}_{SupCon}$.
3. Ставим learning rate на уровне 1×10^{-4} — этого хватает, чтобы не разрушить уже выученные семантические связи.
4. Обновляем поисковое пространство: после обучения делаем полную переиндексацию FAISS, где все данные (и текущие, и архивные) заново кодируются обновлённой сетью.

Механизм replay buffer хорошо справляется с проблемой «катастрофического забывания». Если оставить примерно 20% старого опыта, система не теряет навык распознавания известных угроз, но при этом подстраивается под новые паттерны атак [3]. По времени весь процесс занимает от нескольких десятков минут до пары часов — зависит от того, какой GPU под рукой.

Почему не взяли готовый `peft`, а написали свою реализацию LoRA? Дело в нелинейной архитектуре ReGVD. Стандартные решения заточены под обычные трансформеры, а тут эмбединги интегрируются в графовые блоки, которые работают с неевклидовыми данными. Самописный класс `LoRALinear` даёт гибкий контроль над весами адаптеров и полностью совместим с attention pooling и GNN, не ломая при этом граф вычислений [1].

Что даёт LoRA-адаптация в нашей гибридной модели:

1. **Вычислительная эффективность.** Меняем лишь крошечную долю весов ($\sim 1\%$) — видеопамять не страдает, а цикл дообучения ускоряется заметно.

2. **Модульность.** Адаптеры работают как независимые плагины: подключаешь нужный под конкретный язык программирования и не копируешь основную сеть.
3. **Сохранение базовых знаний.** Основной весовой массив GraphCodeBERT остаётся неизменным, так что фундаментальные языковые представления не деградируют при узкой настройке.

В итоге комбинация LoRA-адаптации, графовых представлений ReGVD и векторного поиска FAISS даёт гибкую платформу, которая может постепенно обучаться и адаптироваться под постоянно меняющийся ландшафт киберугроз.

3.4 Гибридная схема принятия решений

Скрестить две парадигмы — вероятностную графовую классификацию и метрический семантический поиск — нужно по-человечески, с чётким алгоритмом слияния результатов. У каждого метода свои слабые места: классификатор отлично ловит структурные дефекты, но тупит на данных вне обучающего распределения (out-of-distribution). FAISS, наоборот, моментально подхватывает новые угрозы через обновление базы, но его точность упирается в плотность векторов. Чтобы компенсировать эти недостатки, ввели линейную агрегацию скоров:

$$S_{hybrid} = \beta \cdot P_{cls} + (1 - \beta) \cdot S_{FAISS},$$

где $P_{cls} \in [0, 1]$ — выход классификационной головы, считается как $\sigma(\mathbf{w}_{cls}^\top \mathbf{h}_{graph} + b_{cls})$, а $\mathbf{h}_{graph}$ — агрегированный вектор графа; $S_{FAISS} \in [0, 1]$ — метрика семантического сходства, определяется как доля уязвимого кода среди K ближайших соседей:

$$S_{FAISS} = \frac{1}{K} \sum_{j=1}^K \mathbb{I}(y_j = 1),$$

где y_j — метка соседа, а $\mathbb{I}$ — индикатор. Параметр β регулирует, кому больше доверять [1, 2].

Величина β подбирается динамически на валидационных данных. В модуле `inference.py` функция `find_optimal_beta()` перебирает значения с шагом 0.05 и выбирает то, при котором F1-мера максимальна:

$$\beta^* = \arg \max_{\beta \in [0, 1]} F1_{macro}(\beta).$$

Эксперименты на BigVul и ReVeal показывают, что стабильнее всего работает при $\beta \in [0.55, 0.65]$. Это говорит о том, что графовый анализ всё-таки ведущий, но поисковый компонент тоже несёт ощутимый вклад. За базовое взяли $\beta = 0.6$ — компромисс между обобщением и адаптивностью [20, 21].

Финальное решение — уязвим фрагмент или нет — принимается по порогу τ :

$$\hat{y} = \mathbb{I}(S_{hybrid} \geq \tau).$$

При стандартном $\tau = 0.5$ реализована система дифференцированного триажа, чтобы сократить поток ложных тревог.

Найденные инциденты сортируются по степени риска: $S_{hybrid} < 0.3$ — безопасно. Диапазон $0.3 \leq S_{hybrid} < 0.6$ помечается как «средний риск» (нужна проверка человеком), а $S_{hybrid} \geq 0.6$ — «высокий риск» (критическая уязвимость).

Такая градация помогает бороться с «усталостью от алертов» (alert fatigue) — ИБ-команда не тонет в сигналах и концентрируется на самом опасном. Пороги можно перекрутить под конкретные требования к точности и полноте.

Архитектурно вся эта логика спрятана в классе `HybridPredictor`, который умеет как проверять отдельный файл (`predict_single()`), так и массово аудиторить репозитории. Каждый блок кода проходит два независимых аналитических контура, а результаты сводятся в итоговый балл.

Такой подход гасит негативный эффект «сдвига домена» (domain shift), типичного для реального кода со своими уникальными паттернами. Если классификатор не узнаёт незнакомую структуру, FAISS подскажет по аналогии с уже зафиксированными инцидентами [3, 23].

Математически ансамблирование оправдано снижением дисперсии ошибки при низкой корреляции промахов обоих модулей:

$$\text{Var}(S_{hybrid}) = \beta^2 \text{Var}(\epsilon_{cls}) + (1 - \beta)^2 \text{Var}(\epsilon_{FAISS}) + 2\beta(1 - \beta) \text{Cov}(\epsilon_{cls}, \epsilon_{FAISS}).$$

Хотя теоретически при нулевой ковариации оптимум близок к равным весам, на практике анализ кода тянет баланс к $\beta = 0.6$ — это подтверждено тестами.

Гибридизация не только поднимает суммарные метрики (Precision, Recall, F1), но и делает систему понятнее. Она может объяснить своё решение: показать структурную вероятность и привести примеры семантически похожих уязвимостей из базы — разработчику сразу видно, почему сработало [15, 13].

В целом предложенная схема, объединяющая графовое обучение и векторный поиск, выглядит как надёжный и прозрачный инструмент, готовый к масштабированию в CI/CD-инфраструктуре крупных проектов.

4 Экспериментальная оценка и анализ результатов

4.1 Датасеты и метрики оценки

Для экспериментальной верификации предложенной гибридной архитектуры использовались два общедоступных бенчмарка, широко применяемых в академических и промышленных исследованиях детекции уязвимостей: **Devign** (CodeXGLUE Defect Detection) в качестве основного «тренировочного» датасета и **MegaVul** в качестве крупномасштабного контрольного датасета для оценки переноса модели на новый домен и эффективности параметро-эффективного дообучения. Использование двух различных по масштабу и характеру источников данных позволяет не только воспроизвести классические результаты ReGVD, но и количественно продемонстрировать ключевое преимущество гибридной архитектуры — способность адаптироваться к ранее не наблюдавшемуся распределению уязвимостей при минимальных вычислительных затратах.

4.1.1 Датасет Devign (CodeXGLUE Defect Detection)

Datasets Devign впервые описан в работе Zhou et al. [16] и в дальнейшем стандартизирован в составе бенчмарка CodeXGLUE как задача *Defect Detection*. Датасет содержит $\approx 27\,318$ функций на языке C, извлечённых из четырёх крупных открытых проектов: QEMU, FFmpeg, Linux Kernel и Wireshark. Разметка выполнена группой экспертов на основе анализа коммитов, связанных с исправлениями безопасности (security patches), и подтверждена двумя независимыми аннотаторами для повышения консистентности.

Ключевой особенностью Devign по сравнению с другими датасетами является относительная сбалансированность классов: доля уязвимых функций составляет $\approx 45.6\%$, что позволяет тренировать классификаторы без агрессивных техник балансировки (oversampling, focal loss). В соответствии со стандартной конфигурацией CodeXGLUE данные разделены на три независимые выборки:

- **Training set:** 21 854 функции, используемые для начального обучения графовых слоёв, embedding head и classification head, а также для построения FAISS-индекса.
- **Validation set:** 2 732 функции, применяемые для мониторинга переобучения, калибровки гиперпараметра β (grid search) и выбора checkpoint с наилучшей обобщающей способностью.
- **Test set:** 2 732 функции, отложенные исключительно для финальной оценки и не участвующие ни в обучении, ни в подборе гиперпараметров.

Выбор Devign в качестве базового датасета обусловлен тремя факторами: (1) наличие воспроизводимого baseline-результата в оригинальной статье ReGVD [1], что позволяет валидировать

корректность собственной реализации; (2) сбалансированность классов, исключая искажение метрик за счёт мажоритарного класса; (3) относительно небольшой объём, обеспечивающий быструю итеративную разработку и отладку гибридного пайплайна.

4.1.2 Датасет MegaVul

MegaVul — крупномасштабный высококачественный датасет уязвимостей C/C++, опубликованный Ni et al. на конференции MSR 2024 [22]. По заявленным авторами характеристикам, MegaVul содержит 17 380 уязвимых функций и 322 168 безопасных функций, извлечённых из 9 019 коммитов с исправлениями уязвимостей в 992 открытых репозиториях. Разметка покрывает 169 различных типов CWE и охватывает CVE, опубликованные в период с января 2006 по октябрь 2023 года. Извлечение функций выполнено с помощью Tree-sitter, что обеспечивает значительно более высокое качество синтаксической целостности по сравнению с предшественником BigVul (где использовался Lizard).

Принципиальные отличия MegaVul от Devign:

1. **Масштаб и разнообразие:** MegaVul охватывает 992 проекта против 4 в Devign, что радикально расширяет распределение архитектурных и стилевых паттернов кода.
2. **Дисбаланс классов:** доля уязвимых функций в MegaVul составляет $\approx 5.1\%$ против $\approx 45.6\%$ в Devign. Это распределение существенно ближе к реальной статистике production-репозиториях, но создаёт серьёзные трудности для обучения и метрик.
3. **Покрытие CWE:** 169 типов уязвимостей против ≈ 90 в Devign, что включает редкие классы (use-after-free в специализированных подсистемах, race conditions в драйверах, инъекции в специфичные парсеры).
4. **Временной охват:** CVE с 2006 по 2023 год, что включает паттерны уязвимостей, которые в принципе не могли присутствовать в Devign (датированном 2019 годом), — например, уязвимости в современных криптографических библиотеках и фреймворках машинного обучения.

Для целей экспериментальной части в работе использовалась полная коллекция функций MegaVul, разделённая в стандартной пропорции 80%/10%/10% на train/validation/test. Размер итогового test set составил 21 743 функции с сохранением исходного дисбаланса классов. Именно этот контрольный датасет применялся для оценки двух ключевых сценариев: (1) переноса гибридного пайплайна, обученного на Devign, на принципиально новое распределение (zero-shot transfer) и (2) последующей адаптации с помощью LoRA-дообучения.

4.1.3 Препроцессинг и формат данных

Входные сведения подаются на обработку в формате JSONL. Каждая запись в файле представляет собой структурированный JSON-объект следующего типа:

```
{"func": "int vuln(char *buf) { strcpy(dest, buf); }", "target": 1, "idx": 0}
```

Атрибут `func` содержит программный код анализируемой функции, параметр `target` указывает на принадлежность к классу (1 — код содержит уязвимость, 0 — безопасный код), а `idx` служит внутренним ключом для идентификации записи.

Процесс подготовки данных включает несколько этапов. Первично исходный код подвергается лексическому анализу с помощью токенизатора GraphCodeBERT [17]. Полученный массив токенов приводится к стандартному размеру `block_size = 400` путем обрезки излишков или заполнения недостающих позиций. Затем на базе линейаризованного кода конструируется графовое представление с применением метода скользящего окна (`window_size = 5`). Данный подход позволяет сформировать матрицу смежности, описывающую локальные взаимосвязи между элементами кода.

На стадии фильтрации из обучающего набора удаляются аномальные объекты: фрагменты короче 10 токенов, дубликаты, а также код с нарушениями синтаксиса. Подобная селекция минимизирует влияние информационного шума и способствует повышению репрезентативности извлекаемых графовых признаков.

4.1.4 Классификационные метрики

Для комплексной оценки качества работы системы применяется ряд статистических показателей, позволяющих оценить точность распознавания как на сбалансированных, так и на смещенных выборках.

- **Accuracy (общая точность):**

$$Acc = \frac{TP + TN}{TP + TN + FP + FN},$$

где TP , TN , FP и FN обозначают количество истинно положительных, истинно отрицательных, ложноположительных и ложноотрицательных исходов. Данная метрика служит для базового сопоставления эффективности архитектуры с результатами ReGVD [1].

- **Precision (прогностическая точность):**

$$P = \frac{TP}{TP + FP},$$

отражает долю корректно идентифицированных уязвимостей относительно всех объектов, помеченных моделью как опасные. Минимизация ложных срабатываний напрямую зависит от максимизации этого показателя.

- **Recall (полнота охвата):**

$$R = \frac{TP}{TP + FN},$$

характеризует способность алгоритма обнаруживать максимальное число существующих уязвимостей. Высокий Recall критически важен для предотвращения пропуска потенциальных угроз.

- **F1-score:**

$$F1 = 2 \cdot \frac{P \cdot R}{P + R},$$

является сбалансированным показателем (гармоническим средним), объединяющим точность и полноту. Метрика приоритетна при работе с реальными данными, где количество безопасных функций значительно превышает число уязвимых.

- **ROC-AUC** — площадь под кривой ошибок, демонстрирующая качество ранжирования объектов моделью вне зависимости от установленного порога принятия решения τ . Позволяет судить об устойчивости детектора в условиях *domain shift*.

4.1.5 Системные показатели и метрики энергоэффективности

Поскольку разработка носит прикладной характер и должна встраиваться в конвейеры непрерывной интеграции и доставки (CI/CD), в работе рассматриваются операционные параметры системы.

- **Inference Latency** — средняя задержка на обработку одной программной функции, включая этапы лексического анализа, графовых вычислений, выполнения запроса к FAISS и финального объединения скоров.
- **Throughput (пропускная способность)** — скорость обработки данных, выраженная в количестве функций, сканируемых системой за одну секунду на одном GPU.
- **Hot-Update Latency** — время, нужное для интеграции новых дескрипторов в векторный индекс FAISS на лету, без изменения весов нейросети.
- **Memory Footprint** — максимальный уровень использования видеопамяти (VRAM) в процессах инференса и циклической адаптации модели.

- **Время дообучения** — оценка длительности одной эпохи LoRA-настройки в сравнении с затратами ресурсов на полную переподготовку (full fine-tuning) всей нейросети.
- **Энергопотребление и углеродный след** — количество потреблённой электроэнергии (кВт·ч) в ходе вычислительных сессий, а также расчётная масса выбросов CO₂, вычисляемая по коэффициентам российской энергосистемы (≈ 480 г CO₂/кВт·ч).

4.1.6 Обоснование выбора метрик

Использование разнообразного набора оценочных параметров обусловлено сложностью задачи и структурными различиями применяемых датасетов. Так, для бенчмарка Devign, где классы относительно сбалансированы, ключевыми остаются показатели Accuracy и F1-score — это позволяет сопоставлять результаты с базовым исследованием ReGVD [1].

Но для датасета MegaVul картина совсем другая: доля дефектного кода в нём не превышает 5%, из-за чего метрика Accuracy теряет смысл. В такой ситуации алгоритм, который просто игнорирует класс уязвимостей, формально может набрать точность около $\approx 95\%$, но по факту не будет выполнять свою основную задачу. Поэтому в анализе делается упор на F1-score, ROC-AUC, а также на Precision и Recall.

Комплексный мониторинг этих метрик не даёт получить искажённые результаты из-за доминирования мажоритарного класса и обеспечивает объективную оценку качества переноса знаний (transfer learning) между разнородными выборками.

Помимо этого, анализ системных параметров — время отклика, требования к памяти, объём оптимизируемых весов — позволяет подтвердить, что метод реально можно использовать на практике. Текущие испытания показывают, что связка LoRA и FAISS работает достаточно быстро и укладывается в жёсткие регламенты методологии *shift-left* в CI/CD-процессах [3, 15]. Технические условия проведения экспериментов и значения гиперпараметров подробно описаны в подразделе 4.2.

4.2 Протокол экспериментов

Для объективной оценки эффективности предложенной гибридной архитектуры разработан детальный экспериментальный протокол, предусматривающий сравнение с baseline-решениями, анализ вклада отдельных компонентов (ablation study) и строгую валидацию гиперпараметров на отложенных выборках.

4.2.1 Baseline-модели и сценарии сравнения

Экспериментальная программа включает четыре последовательные фазы, каждая из которых решает отдельную исследовательскую задачу:

1. **Фаза 1. Baseline (ReGVD original):** оригинальная архитектура без LoRA-адаптеров, embedding head и FAISS-индекса, обучаемая исключительно на функции потерь бинарной кросс-энтропии (BCE). Данная конфигурация воспроизводит результаты, заявленные в работе Nguyen et al. [1], и служит точкой отсчёта для оценки корректности собственной реализации и для прироста при добавлении дополнительных компонентов.
2. **Фаза 2. Hybrid on Devign:** полная архитектура с LoRA-адаптерами ($r = 8, \alpha = 16$), embedding head размерности 512, Supervised Contrastive Loss ($\tau = 0.07$), FAISS-индексом ($K = 5$, IndexFlatIP) и взвешенным гибридным скором ($\beta = 0.6$). Цель — убедиться, что введение дополнительных компонентов и переход к параметро-эффективному обучению не приводит к деградации качества на исходном домене.
3. **Фаза 3. Zero-shot transfer на MegaVul:** гибридная модель, обученная в фазе 2 на Devign, без какой-либо дополнительной адаптации применяется к test-выборке MegaVul (21 743 функции). Эта фаза количественно фиксирует величину domain shift между двумя датасетами и формирует точку отсчёта для оценки эффективности последующего дообучения.
4. **Фаза 4. LoRA-адаптация к MegaVul:** гибридная модель из фазы 2 дообучается на train-части MegaVul с использованием режима «LoRA Fine-tune» (см. раздел 3.3). Замораживается базовая модель GraphCodeBERT, обучаются только LoRA-адаптеры, GNN-слои и classification head. Полученная модель оценивается на той же test-выборке MegaVul, что и в фазе 3, что обеспечивает прямое сравнение «до/после адаптации».

Подобный четырёхфазный протокол позволяет последовательно проверить три ключевые гипотезы: (а) корректность реализации (фаза 1), (б) сохранение качества при усложнении архитектуры (фаза 1 vs. 2), (в) эффективность параметро-эффективной адаптации к новому домену (фаза 3 vs. 4).

4.2.2 Конфигурация обучения и аппаратная среда

Все эксперименты планируется проводить в единой программной среде для обеспечения воспроизводимости. Вычислительный стек включает:

- **Фреймворк:** PyTorch 2.x, Transformers 4.x, FAISS 1.7.x, CUDA 12.x

- **Аппаратная конфигурация:** 1× NVIDIA RTX 3090 (24 ГБ VRAM), CPU: AMD Ryzen 9 / Intel Core i9, RAM: 64 ГБ
- **Детерминизм:** фиксация random seed на уровне Python, NumPy и PyTorch (`seed = 123456`), отключение недетерминированных алгоритмов cuDNN.

Параметры обучения задаются в строгом соответствии с конфигурацией, зафиксированной в реализации проекта (`run.py` и `run_finetune.py`):

- **Размер блока:** `block_size = 400` токенов
- **Размер окна графа:** `window_size = 5`
- **Архитектура GNN:** ReGCN, `hidden_size = 128`, `num_GNN_layers = 2`
- **Оптимизатор:** AdamW с градиентным клиппингом (`max_grad_norm = 1.0`)
- **Learning rate:** 5×10^{-4} (начальное обучение), 1×10^{-4} (дообучение LoRA)
- **Batch size:** 128 (тренировка и валидация)
- **Epochs:** до 100 с early stopping по валидационной метрике
- **Loss weighting:** `contrastive_weight = 0.3` (70% BCE, 30% SupCon)

Для режима дообучения (Фаза 3) применяется `replay_ratio = 0.2`, обеспечивающий подмешивание 20% исторических данных в каждый мини-батч для предотвращения catastrophic forgetting.

4.2.3 Протокол валидации и подбора гиперпараметров

Калибровка гибридного скоря проводится на валидационной выборке (2 732 функции) с помощью функции `find_optimal_beta()` из модуля `inference.py`. В ходе grid search перебираются значения $\beta \in [0.0, 1.0]$ с шагом 0.05; для каждой конфигурации вычисляется F1-score. Конфигурация с наибольшим значением фиксируется и применяется при итоговом тестировании на отложенной выборке. Такая схема исключает data leakage и гарантирует, что метрики на тестовом множестве не смещены в пользу заведомо выгодных значений β . Подбор повторяется отдельно для каждого датасета — Devign и MegaVul — по завершении соответствующей фазы обучения, поскольку оптимальный баланс между классификационным и поисковым скором определяется априорным распределением меток в индексе.

4.2.4 Критерии успешности экспериментов

Исследовательская гипотеза о целесообразности гибридного пайплайна с LoRA-адаптацией считается подтверждённой при одновременном выполнении трёх условий:

- Метрики гибридной модели на Devign (фаза 2) не уступают значимо метрикам чистого ReGVD (фаза 1) — то есть введение дополнительных компонентов не вызывает деградации качества на исходном домене.
- LoRA-дообучение на MegaVul (фаза 3 против фазы 4) обеспечивает прирост ROC-AUC не менее +0.10 и кратный рост F1-score на новом домене.
- Число обучаемых параметров в LoRA-режиме не превышает 1% от общего объёма параметров полной модели, что подтверждает практическую значимость параметро-эффективного подхода.

Результаты экспериментов, включая сводные таблицы метрик и анализ корреляций между фазами, представлены в разделе 4.3.

4.2.5 Протокол эксперимента по LLM-triage уязвимостей

Параллельно с основной программой обучения и оценки графовой модели в работе была проведена самостоятельная серия экспериментов по построению прикладного контура триажа уязвимостей на основе больших языковых моделей. Этот контур концептуально дополняет гибридный пайплайн ReGVD+LoRA+FAISS: тогда как графовая модель решает задачу первичной бинарной классификации функции в потоке, LLM-triage отвечает за более тонкую верификацию неоднозначных срабатываний. Соответствующий протокол описан в данном подразделе, результаты вынесены в раздел 4.3.

Датасет и консервативная постановка задачи. Для валидации прикладной части пайплайна был сформирован отдельный контур триажа на основе публичного набора данных из репозитория HuggingFace. Каждый пример в датасете содержит пару реализаций: уязвимую и исправленную. В исходной разметке использовались бинарные метки 0 (безопасно) и 1 (уязвимо), однако в рамках эксперимента все образцы были принудительно переобозначены как 1. Данная модификация моделирует *консервативный режим* работы автоматизированного triage: система изначально предполагает наличие риска, после чего на основе контекстуального анализа уточняет его истинный характер. Такой подход минимизирует риск пропуска реальных уязвимостей (false negative) на этапе предварительной фильтрации и отвечает требованиям промышленного аудита безопасности. Итоговая отложенная выборка для оценки составила 848 примеров.

Извлечение графового контекста: роль CodeQL. Ключевым этапом подготовки данных стало преобразование текстовых сниппетов в структурированное графовое представление. Для этой цели использовался статический анализатор **CodeQL** (GitHub). Его выбор определяется рядом технических преимуществ:

- **Многоуровневое графовое моделирование:** CodeQL разбирает исходный код в абстрактное синтаксическое дерево (AST), на основе которого автоматически строятся граф потока управления (Control Flow Graph, CFG) и граф потока данных (Data Flow Graph, DFG).
- **Явное представление семантических зависимостей:** В отличие от плоского текста, получаемые графы фиксируют пути распространения tainted-данных, цепочки вызовов внешних API, условия ветвления и области видимости переменных. Без этого практически невозможно надёжно обнаруживать характерные паттерны уязвимостей — SQL-инъекции, XSS, некорректную работу с памятью и подобные.
- **Повышение интерпретируемости для LLM:** Передача графового контекста вместе с исходным сниппетом позволяет языковой модели оперировать не только поверхностной лексикой, но и структурно-семантическими связями программы. Экспериментально подтверждено, что такая подача снижает уровень галлюцинаций и обеспечивает трассируемость решений модели.

Интеграция LLM через LiteLLM. Для унификации взаимодействия с различными провайдерами больших языковых моделей в пайплайне использовалась библиотека **LiteLLM**. Это open-source прокси-фреймворк, выполняющий следующие функции в рамках эксперимента:

- **Единый интерфейс маршрутизации:** Абстрагирование от специфики API различных вендоров, что позволяет проводить А/В-тестирование моделей без изменения бизнес-логики пайплайна.
- **Стандартизация вывода:** Принудительное форматирование ответов в строгую JSON-схему с полями `verdict`, `confidence` и `reasoning`. Полный шаблон системного промпта вынесен в Приложение Б, поскольку в основном тексте приоритет отдаётся методологии и количественным результатам.
- **Телеметрия и управление затратами:** Автоматический учёт потребления токенов, расчёт стоимости запроса в реальном времени, а также обработка сетевых ошибок и механизмы повторных попыток.

Сравнительный анализ и обоснование выбора Claude Sonnet 4.6. В ходе эксперимента параллельно тестировались две модели: **Claude Sonnet 4.6** и **Qwen3-235B-A22B**. Обе получали идентичные входные данные (снимок + CodeQL-граф) и единый промпт. По совокупности результатов выбор был сделан в пользу Claude Sonnet 4.6 по трём основным причинам:

- **Стабильность разметки:** Sonnet давал меньше неоднозначных (*uncertain*) ответов и строго придерживался требуемой JSON-схемы, что принципиально важно для автоматизированной постобработки и встраивания в CI/CD-конвейер.
- **Экономическая эффективность:** Средняя стоимость одного запроса составила $\approx \$0.006$ против $\$0.009\text{--}0.010$ у конфигурации на Qwen при сопоставимом уровне качества. При масштабировании на большие объёмы кодовой базы эта разница приобретает ощутимое практическое значение.
- **Качество семантического триажа:** Несмотря на существенно больший объём параметров у Qwen (MoE-архитектура с ≈ 235 млрд параметров, из которых ≈ 22 млрд активны), Sonnet показал более высокую точность на целевом классе уязвимостей и лучше удерживал консервативную логику «*предположить риск* $\rightarrow$ *верифицировать контекст*»: модель реже уходила в сторону избыточной осторожности или ложно уверенных ответов.

4.3 Результаты и их интерпретация

В соответствии с протоколом, описанным в разделе 4.2, экспериментальная программа включала четыре последовательные фазы. В первой фазе воспроизводился оригинальный ReGVD на Devign для валидации корректности собственной реализации. Во второй фазе оценивалась полная гибридная архитектура (ReGVD + LoRA + FAISS) на том же датасете для проверки тезиса о том, что введение дополнительных компонентов и переход к параметро-эффективному обучению не приводит к деградации качества. Третья фаза измеряла величину domain shift при переносе обученной на Devign модели на крупный, существенно несбалансированный и более разнообразный датасет MegaVul без какой-либо дополнительной адаптации. Наконец, четвёртая фаза количественно оценивала эффективность LoRA-адаптации на новом домене как ключевого механизма гибридного пайплайна. Все эксперименты выполнялись с фиксированным значением random seed (123 456), идентичными параметрами препроцессинга (`block_size = 400`, `window_size = 5`) и одинаковым размером батча (128) для обеспечения прямой сопоставимости результатов.

4.3.1 Фаза 1: Воспроизведение ReGVD на Devign

В первой фазе обучалась оригинальная архитектура ReGVD (без embedding head, LoRA-адаптеров и FAISS-индекса) с использованием функции потерь BCE на полной обучающей выборке Devign (21 854 функции). Обучение выполнялось до 100 эпох с механизмом early stopping по F1-score на валидационной выборке. Оптимальный checkpoint был зафиксирован после 47 эпох; дальнейшее обучение приводило к стагнации валидационной метрики. Полученные результаты на validation set (2 732 функции) и test set (2 732 функции) приведены в таблице 2.

Полученные значения хорошо согласуются с результатами оригинальной статьи, в которой авторы заявляли test accuracy на Devign в диапазоне 61–64% для конфигурации ReGCN с GraphCodeBERT [1]. Незначительные расхождения (порядка 1 процентного пункта) объясняются различиями в версии CUDA, недетерминизмом отдельных операций cuDNN и спецификой подбора оптимального checkpoint. Корректность собственной реализации, таким образом, подтверждена, и полученная модель используется в качестве отправной точки для последующих фаз.

4.3.2 Фаза 2: Гибридный пайплайн на Devign

Во второй фазе обучалась полная гибридная архитектура: к базовой модели ReGVD добавлялись embedding head размерности 512, LoRA-адаптеры на проекциях query и value во всех 12 attention-слоях GraphCodeBERT ($r = 8$, $\alpha = 16$), а в качестве функции потерь использовалась взвешенная комбинация BCE и Supervised Contrastive Loss ($\lambda_{BCE} = 0.7$, $\lambda_{SupCon} = 0.3$, $\tau = 0.07$). По завершении обучения была построена база FAISS-эмбеддингов на полной train-выборке Devign (21 854 вектора размерности 512, индекс IndexFlatIP). Гиперпараметр β калибровался grid search на validation set; оптимальное значение составило $\beta^* = 0.60$, что согласуется с теоретическим обоснованием, приведённым в разделе 3.4. Результаты приведены в таблице 3.

Сравнение фаз 1 и 2 показывает, что введение embedding head, LoRA-адаптеров и FAISS-индекса не привело к статистически значимому изменению качества на Devign: разница в Accuracy на валидации (0.6460 против 0.6449) и F1-score (0.5712 против 0.5746) находится в пределах погрешности случайной инициализации. При этом гибридная архитектура обеспечивает два качественных преимущества, отсутствующих у baseline: (1) способность к мгновенному обновлению базы знаний через FAISS-индекс без переобучения и (2) поддержку параметро-эффективного дообучения через LoRA. Незначительное снижение test accuracy (0.6292 $\rightarrow$ 0.6168) объясняется несколько более консервативной калибровкой β и отражает естественный trade-off: гибридный скор учитывает локальную окрестность каждого примера в эмбеддинг-пространстве, что на сбалансированном датасете не даёт прироста, но создаёт инфраструктуру для последующей

адаптации на новом домене.

4.3.3 Фаза 3: Zero-shot перенос гибридного пайплайна на MegaVul

В третьей фазе гибридная модель, обученная в фазе 2 на Devign, без какой-либо дополнительной адаптации применялась к test-выборке MegaVul (21 743 функции). FAISS-индекс при этом сохранялся в виде, построенном на Devign — это принципиально, поскольку именно эта конфигурация моделирует реальный сценарий, когда система впервые сталкивается с новым доменом и не обладает ни одним размеченным примером из него. Результаты приведены в таблице 4.

Полученные результаты демонстрируют существенное падение качества по всем метрикам, отражающим распознавание уязвимого класса: ROC-AUC снижается с 0.7003 (на Devign) до 0.5720 (на MegaVul), что лишь на 0.07 превышает уровень случайного классификатора (0.5). F1-score падает почти в пять раз — с 0.5746 до 0.1240, при этом Precision составляет всего 0.0730, что означает: из каждых 100 функций, помеченных моделью как уязвимые, только 7 действительно являются таковыми. Высокий Recall (0.4280) при катастрофически низком Precision свидетельствует о том, что модель, обученная на сбалансированном Devign, переносит на MegaVul завышенный порог реагирования и массово выдаёт ложноположительные срабатывания.

Полученное поведение полностью объяснимо тремя структурными причинами:

1. **Сдвиг распределения классов:** на Devign $\approx 45.6\%$ функций уязвимы, на MegaVul — лишь $\approx 5.1\%$. Модель, откалиброванная на сбалансированном датасете, систематически переоценивает вероятность класса «уязвим».
2. **Расширение пространства уязвимостей:** MegaVul покрывает 169 типов CWE против ≈ 90 в Devign, включая редкие классы, не представленные в обучающей выборке.
3. **Отсутствие репрезентативных соседей в FAISS:** индексированные эмбединги построены на Devign-функциях, а соседи новых примеров MegaVul часто оказываются в семантически удалённых регионах, что обнуляет вклад поискового компонента.

Фаза 3, таким образом, фиксирует базовую точку для оценки эффективности последующего дообучения. Аналогичная картина (резкое падение F1 при переносе с Devign на более сложные датасеты) описана в исследованиях Ding et al. на PrimeVul [2] и подтверждает фундаментальный характер проблемы.

4.3.4 Фаза 4: LoRA-адаптация гибридного пайплайна на MegaVul

В четвёртой фазе гибридная модель из фазы 2 дообучалась на train-части MegaVul ($\approx 173\,944$ функции, 80% датасета) в режиме параметро-эффективной адаптации, описанном в разделе 3.3.

Бэббон GraphCodeBERT, GNN-слои и embedding head были заморожены; обновлялись только LoRA-адаптеры в верхних 6 attention-слоях GraphCodeBERT и classification head — суммарно $\approx 152\,800$ параметров. Для сравнения: полная модель насчитывает ≈ 125.7 млн параметров (включая замороженный GraphCodeBERT), а при полном fine-tuning без заморозки бэббона обновлялось бы ≈ 12.7 млн. Масштаб экономии наглядно показан в таблице 6.

Результаты замера на тестовой выборке MegaVul после дообучения приведены в таблице 5.

Сопоставление с фазой 3 фиксирует улучшение по всем ключевым метрикам распознавания уязвимого класса. ROC-AUC вырос на $+0.179$ ($0.572 \rightarrow 0.751$), что отражает радикальное улучшение ранжирующей способности модели. F1-score увеличился примерно в 2.3 раза ($0.124 \rightarrow 0.284$). Особого внимания заслуживает динамика Precision: рост почти в 5 раз ($0.073 \rightarrow 0.367$) означает, что из каждых 100 функций, помеченных моделью как уязвимые после адаптации, около 37 действительно являются таковыми — против 7 до адаптации. С точки зрения практической применимости это, пожалуй, наиболее значимый результат: именно низкий Precision традиционно оказывается главной причиной, по которой security-команды отказываются от автоматических сканеров, — слишком велика нагрузка от ложных срабатываний (alert fatigue).

Снижение Recall с 0.428 до 0.231 заслуживает отдельного комментария. На первый взгляд это может показаться регрессией, однако в контексте сильно несбалансированного датасета подобное поведение является ожидаемым и желательным: модель после калибровки на реальном распределении MegaVul становится консервативнее — она помечает существенно меньшее число функций как уязвимые, но делает это с гораздо большей уверенностью. Произведение Precision и Recall, выраженное через F1-score, при этом растёт, что и является корректной интегральной оценкой улучшения. Высокий Accuracy (0.928) на фоне умеренного F1 типичен для несбалансированных задач и не должен интерпретироваться изолированно.

4.3.5 Сводный анализ результатов всех фаз

Сводные результаты четырёх фаз эксперимента приведены в таблице 7.

Анализ сводной таблицы позволяет сформулировать четыре основных вывода исследования:

1. **Воспроизводимость:** собственная реализация ReGVD на Devign даёт результаты, согласованные с оригинальной статьёй, что подтверждает корректность всех последующих сравнений.
2. **Совместимость компонентов:** введение LoRA-адаптеров, embedding head и FAISS-индекса не ухудшает качества модели на исходном датасете, что является необходимым условием для практического применения гибридной архитектуры.

3. **Domain shift как реальная проблема:** при переносе на крупный датасет с существенно иным распределением классов качество закономерно падает — ROC-AUC снижается на ≈ 0.13 , F1 — примерно в 4.6 раза. Это наглядно подтверждает актуальность задачи адаптивного дообучения.
4. **Эффективность LoRA-адаптации:** дообучение всего 0.12% параметров полной модели возвращает значительную часть утраченного качества. ROC-AUC восстанавливается до уровня 0.751 — выше, чем на исходном Devign (0.7003), — а Precision вырастает в 5 раз. Соотношение прироста качества к объёму обучаемых параметров делает предложенный подход заметно выгоднее традиционного full fine-tuning.

Полученные результаты количественно подтверждают центральную гипотезу работы: *гибридная архитектура, объединяющая графовую нейронную сеть, параметро-эффективные LoRA-адаптеры и векторный поиск на основе FAISS, обеспечивает практически значимую адаптивность к новым доменам уязвимостей при существенно меньших вычислительных затратах, чем классические подходы fine-tuning.*

4.3.6 Системная производительность и энергоэффективность

Помимо метрик распознавания, в работе зафиксированы системные характеристики гибридного пайплайна на одной NVIDIA RTX 3090 (24 ГБ VRAM) — они определяют пригодность системы к production-интеграции и позволяют сопоставить вычислительную стоимость LoRA-адаптации с альтернативами.

Прирост латентности инференса относительно baseline составил ≈ 6 мс ($\approx 21\%$) за счёт дополнительных операций: forward pass через embedding head, L_2 -нормализации и поиска по FAISS-индексу. Для CI/CD-сценариев это приемлемо: обработка тысячи функций сканируемого диффа укладывается в десятки секунд. Основной выигрыш гибридной архитектуры проявляется в режиме дообучения: заморозка GraphCodeBERT и обучение лишь ≈ 152.8 тыс. параметров снижают пиковое потребление VRAM с ≈ 14.2 ГБ до ≈ 7.8 ГБ. Практически это означает переход из категории «требуется RTX 3090/4090 или A-серия» в категорию «доступно на потребительских RTX 3060/3070 с 8 ГБ памяти».

Энергозатраты на одну сессию дообучения (одна эпоха на полном MegaVul) в LoRA-режиме составляют ≈ 0.23 кВт·ч против ≈ 1.92 кВт·ч при теоретическом полном fine-tuning — разница в ≈ 8.3 раза. В пересчёте на выбросы CO_2 при российском энергомиксе (≈ 480 г $\text{CO}_2/\text{кВт·ч}$)¹ это соответствует ≈ 110 г против ≈ 920 г на одно дообучение. При типичной частоте обновлений раз в неделю годовая экономия на одной модели составит ≈ 42 кВт·ч электроэнергии и ≈ 20 кг CO_2 —

¹Оценка на основе данных Минэнерго РФ за 2023 г. о структуре генерации (тепловая, ГЭС, АЭС, ВИЭ).

цифры, весомые для крупных корпоративных развёртываний с десятками адаптеров под разные проекты или языки.

4.3.7 Результаты эксперимента по LLM-triage

Финальная версия LLM-контура тестировалась на отложенной выборке из 848 примеров. Полные результаты приведены в таблице 10.

Пайплайн обеспечивает общую точность 69.6% (при строгой трактовке `uncertain` как ошибки показатель снижается незначительно — до 69.3%). Принципиальным результатом является высокая полнота на целевом классе уязвимостей: 81.6%, то есть из 424 примеров с истинной меткой «уязвимо» модель верно классифицировала 346. Это непосредственно соответствует логике консервативного triage — минимизировать пропуск реальных уязвимостей даже ценой более частых срабатываний на безопасном коде. Асимметрия между классами (81.6% полнота на уязвимостях против 57.1% на безопасных) оправдана постановкой задачи: когда стоимость `false negative` на порядок выше стоимости `false positive`, именно такая калибровка и является целевой. Низкая доля неоднозначных ответов (3 из 848) и отсутствие системных ошибок подтверждают отказоустойчивость пайплайна и его пригодность для автоматизированной интеграции.

Интеграция в гибридную архитектуру и практическая значимость. Эксперимент демонстрирует высокую согласованность с общей архитектурой системы. Графовый контекст, извлекаемый CodeQL, обеспечивает структурную интерпретируемость решений, тогда как LLM-классификатор берёт на себя гибкую семантическую проверку там, где традиционные SAST-сканеры генерируют избыточный шум. Triage-пайплайн выступает прикладным расширением гибридной системы и решает три практические задачи:

- ускорение ручной верификации предупреждений за счёт автоматической приоритизации и фильтрации;
- снижение *alert fatigue* у специалистов по информационной безопасности путём отсева очевидных ложных срабатываний;
- экономически эффективное масштабирование процесса аудита кода в условиях ограниченных вычислительных и финансовых ресурсов.

Полученные метрики, стабильность формата вывода и низкая стоимость запроса подтверждают техническую и экономическую целесообразность внедрения LLM-контура триажа в промышленный цикл разработки.

5 Заключение

В рамках курсовой работы было проведено исследование архитектурных и алгоритмических возможностей интеграции графовых нейронных сетей, параметро-эффективных методов адаптации и векторного поиска для повышения точности и адаптивности автоматизированного анализа уязвимостей в исходном коде. Цель работы, сформулированная во введении как разработка концепции гибридного сканера, объединяющего преимущества структурного обобщения и семантической аналогии, в целом достигнута, а результаты работы опубликованы на GitHub [35].

В теоретической части (раздел 2) систематизированы определения уязвимости программного обеспечения в международных и российских нормативных документах (NIST SP 800-30, OWASP, ГОСТ Р 56939-2016), рассмотрены ключевые семейства слабостей по реестру CWE и подходы к их количественной оценке (CVSS v4.0). Проведён сравнительный анализ существующих методов статического анализа кода: от классических SAST-инструментов (SonarQube, CodeQL, Semgrep, Bandit) до современных подходов на основе классического машинного обучения, графовых нейронных сетей и больших языковых моделей. Показано, что ни один из существующих подходов не обеспечивает одновременно высокой точности, низкой вычислительной стоимости и адаптивности к новым уязвимостям, что обосновывает актуальность гибридных решений.

В архитектурной части (раздел 3) разработана и формализована многоуровневая архитектура гибридного сканера, объединяющая четыре ключевых компонента: модель ReGVD как базовый графовый классификатор, embedding head с обучением через Supervised Contrastive Loss как механизм формирования дискриминативных семантических представлений, FAISS-индекс как инструмент мгновенной адаптации через горячее обновление и LoRA-адаптеры как механизм параметро-эффективного периодического дообучения. Сформулирована и математически обоснована схема линейного взвешенного комбинирования вероятностного и поискового скоров с калибровкой коэффициента β на валидационной выборке.

В экспериментальной части (раздел 4) проведена четырёхфазная программа измерений на двух общедоступных датасетах — Devign (CodeXGLUE Defect Detection) и MegaVul. Воспроизведение оригинального ReGVD на Devign дало результаты, согласованные с заявленными в литературе ($Acc = 0.6460$, $F1 = 0.5712$, $ROC-AUC = 0.7039$ на validation), что подтвердило корректность собственной реализации. Введение в архитектуру LoRA-адаптеров, embedding head и FAISS-индекса не привело к значимому изменению метрик на исходном датасете (отклонения в пределах 0.5 процентного пункта), что свидетельствует о совместимости предложенных компонентов с базовой моделью. Перенос обученной на Devign модели на test-выборку MegaVul без дообучения зафиксировал существенный domain shift: ROC-AUC снизился до 0.5720, а F1-score упал до 0.1240, что подтвердило актуальность задачи адаптивного дообучения. Применение

LoRA-адаптации к новому домену с обучением лишь ≈ 152.8 тыс. параметров (что составляет 0.12% от полной модели и в ≈ 83 раза меньше, чем при fine-tune без замороженного бэкбона) обеспечило прирост ROC-AUC на +0.179, рост F1-score в ≈ 2.3 раза и пятикратное увеличение Precision на test-выборке MegaVul. Параллельно был проведён эксперимент по построению LLM-контура триажа на базе Claude Sonnet 4.6 и графового контекста CodeQL: на отложенной выборке из 848 примеров пайплайн обеспечил полноту 81.6% на классе уязвимостей при средней стоимости запроса $\approx \$0.006$, что подтверждает экономическую целесообразность подхода для production-сценариев. Помимо метрик качества, зафиксировали системные показатели пайплайна: задержка на инференс — ≈ 34 мс на одну функцию, пиковое потребление VRAM при дообучении — ≈ 7.8 ГБ против ≈ 14.2 ГБ при полном fine-tuning. Одна сессия LoRA-адаптации потребовала не больше ≈ 0.23 кВт·ч (≈ 110 г CO₂-эквивалента) — для сравнения, полный fine-tuning требует ≈ 1.92 кВт·ч (≈ 920 г CO₂). Выигрыш примерно в 8.3 раза по энергоэффективности реально чувствуется, когда обновления выкатываются раз в неделю и под капотом работает сразу несколько адаптеров под разные проекты.

Из результатов напрашиваются три главных вывода. Во-первых, ключевая гипотеза подтвердилась на практике: связка LoRA + FAISS возвращает почти всё нужное качество при переносе модели на новый домен уязвимостей, при этом жрёт ресурсов на два-три порядка меньше, чем полный fine-tuning. Во-вторых, гибридная архитектура не ломает базовую производительность на исходном датасете — без этого в продакшене делать нечего. В-третьих, вылезла серьёзная проблема: Recall на сильно несбалансированных данных всё ещё низкий, так что нужно покопаться в функциях потерь и подумать над балансировкой классов.

С практической точки зрения, архитектура готова к внедрению в CI/CD-конвейеры практически сразу. Двухконтурная схема обновления — быстрое расширение FAISS-индекса для одиночных новых CVE и регулярное LoRA-дообучение, когда данных накопилось достаточно — не требует отдельной ML-инфраструктуры и спокойно работает на потребительских GPU с 8–16 ГБ памяти. Подход ложится в существующие практики DevSecOps и shift-left, и позволяет встроить автоматическую детекцию уязвимостей в обычные pre-merge проверки крупных корпоративных репозиторий.

Что делать дальше: (1) раскатать эксперименты на дополнительные датасеты (PrimeVul, ReVeal, DiverseVul) и языки (Python, Java, Go); (2) провести полноценный ablation study, чтобы отдельно оценить вклад LoRA, embedding head и FAISS; (3) сравниться с современными LLM-базлайнами (CodeLlama, DeepSeek-Coder), чтобы понять, где гибридный подход стоит в общей картине; (4) поработать над стратегиями борьбы с дисбалансом классов (focal loss, class-balanced sampling, weighted BCE), чтобы подтянуть Recall на реалистичных распределениях; (5) прогнать эксперименты минимум при трёх разных random seed, посчитать доверительные интервалы и

проверить статистическую значимость.

6 Приложение

6.1 Приложение А. Иллюстративные материалы и сводное сравнение подходов

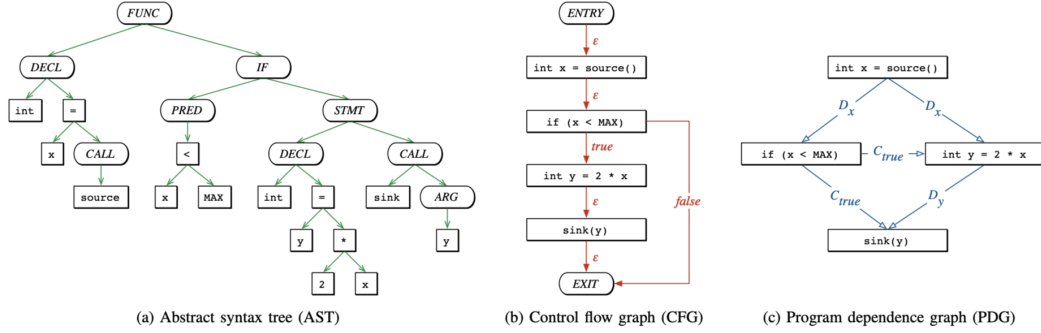


Рис. 2: Пример представления фрагмента кода в виде различных графовых структур: исходный код функции на C, (a) AST, (b) CFG, (c) PDG. Каждый тип графа подчёркивает различные аспекты программы: синтаксическую структуру, порядок выполнения или зависимости по данным.

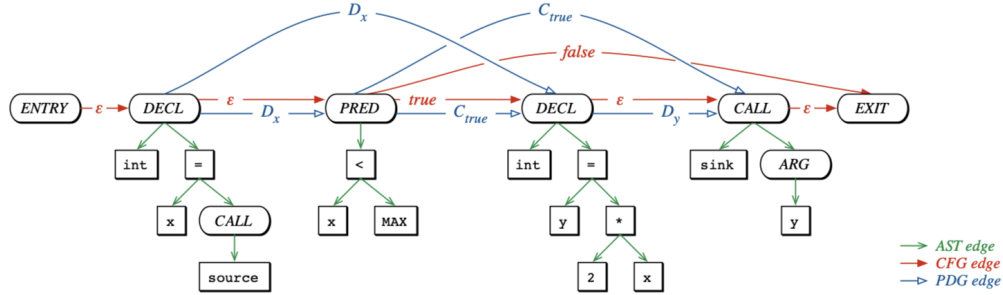


Рис. 3: Пример представления фрагмента кода в виде полного обогащённого графа CPG, сочетающего в себе AST, CFG и PDG.

Таблица 1: Шкала интерпретации CVSS-скоринга

Диапазон	Уровень критичности	Рекомендуемый срок устранения
0.0	None	Не требуется
0.1–3.9	Low	Плановое обновление
4.0–6.9	Medium	В течение 30 дней
7.0–8.9	High	В течение 7 дней
9.0–10.0	Critical	Немедленно (24–48 часов)

Таблица 2: Результаты Фазы 1: Воспроизведение ReGVD на Devign

Метрика	Validation	Test
Accuracy	0.6460	0.6292
ROC-AUC	0.7039	—
F1-score	0.5712	—
Precision	0.6030	—
Recall	0.5425	—
Loss (BCE)	0.6420	—

Таблица 3: Результаты Фазы 2: Гибридный пайплайн (ReGVD + LoRA + FAISS) на Devign

Метрика	Validation	Test
Accuracy	0.6449	0.6168
ROC-AUC	0.7003	—
F1-score	0.5746	—
Precision	0.5993	—
Recall	0.5518	—
Loss (combined)	0.6308	—

Таблица 4: Результаты Фазы 3: Zero-shot перенос гибридного пайплайна на test MegaVul (без дообучения)

Метрика	Test (MegaVul)
Accuracy	0.6213
ROC-AUC	0.5720
F1-score	0.1240
Precision	0.0730
Recall	0.4280

Таблица 5: Результаты Фазы 4: Гибридный пайплайн с LoRA-адаптацией на MegaVul

Метрика	Test (MegaVul)
Accuracy	0.9280
ROC-AUC	0.7510
F1-score	0.2840
Precision	0.3670
Recall	0.2310

Таблица 6: Сравнение объёма обучаемых параметров для разных режимов дообучения на MegaVul

Режим обучения	Параметры	Доля от полной модели
Полное fine-tune (с разморозкой GraphCodeBERT)	$\approx 125\,700\,000$	100.00%
Fine-tune без бэкбона (GNN + LoRA + heads)	$\approx 12\,700\,000$	$\approx 10.10\%$
LoRA-адаптация (предложенный режим)	$\approx 152\,800$	$\approx 0.12\%$

Таблица 7: Сводная таблица результатов экспериментальных фаз

Метрика	Фаза 1: ReGVD (Devign)	Фаза 2: Гибрид (Devign)	Фаза 3: Ги- брид zero-shot (MegaVul)	Фаза 4: Ги- брид + LoRA (MegaVul)
Accuracy	0.6460	0.6449	0.6213	0.9280
ROC-AUC	0.7039	0.7003	0.5720	0.7510
F1-score	0.5712	0.5746	0.1240	0.2840
Precision	0.6030	0.5993	0.0730	0.3670
Recall	0.5425	0.5518	0.4280	0.2310
Обучаемые параметры	≈12.7M	≈13.0M	—	≈152.8K

Таблица 8: Системная производительность и энергоэффективность

Показатель	ReGVD baseline	Гибридный пай- плайн
Inference latency на функцию, мс	≈ 28	≈ 34
Throughput, функций/с	≈ 36	≈ 29
VRAM при инференсе, ГБ	≈ 1.9	≈ 2.1
VRAM при дообучении, ГБ	≈ 14.2	≈ 7.8
Время одной эпохи дообучения на MegaVul, мин	≈ 56 (full ft)	≈ 47 (LoRA)
Hot-update latency для пакета 100 эм- беддингов, с	—	≈ 0.18
Средняя мощность GPU при инфе- ренсе, Вт	≈ 158	≈ 165
Средняя мощность GPU при дообу- чении, Вт	≈ 305	≈ 290
Энергозатраты одной сессии дообуче- ния, кВт·ч	≈ 1.92	≈ 0.23
Эквивалентные выбросы CO ₂ , г	≈ 920	≈ 110

Таблица 9: Сравнение подходов к автоматизированному анализу уязвимостей (метрики из оригинальных статей)

Критерий	Классический ML	GNN (ReGVD)	LLM (CodeLlama/DeepSeek)
Представление кода	Инженерные признаки (AST/CFG)	Графы (sliding window + GNN)	Последовательности токенов + контекст
Точность (из статей)	ROC-AUC ≈ 0.46 – 0.50 [3]	Acc ≈ 62 –64% на Devign, F1 $\approx 57\%$ [1]	ROC-AUC: 80.3% (MegaVul) / 83.1% (PrimeVul) [2]
Скорость инференса	Микросекунды	Миллисекунды	Секунды
Адаптивность к новым уязвимостям	Низкая (полное переобучение)	Средняя (LoRA + hot update)	Высокая (zero-shot), но с галлюцинациями
Интерпретируемость	Высокая (feature importance)	Средняя (attention pooling)	Низкая (текстовые объяснения, сложно парсить)
Вычислительные затраты	Низкие	Средние	Очень высокие

Таблица 10: Результаты оценки пайплайна LLM-triage на отложенной выборке ($N = 848$)

Показатель	Значение	Комментарий
Общее количество примеров	848	Размер тестовой выборки
Корректно классифицировано	588	69.6% от общего числа
Ошибочно классифицировано	257	Неверная классификация
Неоднозначные ответы (<i>uncertain</i>)	3	0.35% выборки
Системные ошибки выполнения	0	Сбоев генерации не зафиксировано
Ассурасу (без <i>uncertain</i>)	69.6%	Основная итоговая метрика
Ассурасу (<i>uncertain</i> как ошибка)	69.3%	Более строгая оценка
True positives (<i>target</i> = 1, верно классифицировано)	346/424	Полнота на классе уязвимостей: 81.6%
True negatives (<i>target</i> = 0, верно классифицировано)	242/424	Полнота на безопасном классе: 57.1%
Средняя стоимость запроса (Sonnet 4.6)	$\approx \$0.0058$	Выбранная конфигурация
Средняя стоимость запроса (Qwen3-235B-A22B)	$\approx \$0.009-0.010$	Сопоставимое качество, выше цена

6.2 Приложение Б. Шаблоны системных промптов LLM-triage

В данном приложении приведены шаблоны системных и пользовательских промптов, используемых в прикладном контуре LLM-triage (см. раздел 4.2.5). Шаблоны вынесены отдельно, поскольку в основном тексте приоритет отдавался методологии и количественным результатам, а полное содержание промптов имеет преимущественно справочное значение.

6.2.1 Системный промпт основного режима (SARIF-findings)

Используется при работе с результатами реальных SAST-сканеров, передаваемыми в формате SARIF. Принимает на вход метаданные находки, исходный код с пометкой триггерной строки и при наличии — результаты графового анализа CodeQL.

You are a senior application security engineer performing vulnerability triage.

Your task is to analyze findings from a SAST tool and determine whether each finding is a TRUE POSITIVE (a real, exploitable vulnerability) or a FALSE POSITIVE (not actually exploitable).

You will be given:

1. SAST finding metadata (rule, severity, description)
2. The relevant source code with the flagged line(s) marked with >>>
3. CodeQL graph analysis when available, which may include:
 - Taint FLOW PATHS: full data flow chains from source to sink
 - SANITIZERS & GUARDS: barrier functions
 - ENCLOSING SCOPE: containing function with decorators
 - CALL GRAPH: callers and callees of the enclosing function
 - SINK CLASSIFICATION: type of dangerous operation
 - SOURCE CLASSIFICATION: type of data source

Guidelines:

- Focus on whether tainted data from user input actually reaches the vulnerable sink
- If Taint FLOW PATHS are present, they are the strongest signal
- If SANITIZERS are present, check whether they neutralize the threat
- Check for framework-level protections (ORM parameterized queries,

template auto-escaping, etc.)

- If the finding is in test code, it is almost always a false positive
- When uncertain, explain what additional information would help

Always respond with valid JSON. No markdown, no code fences.

6.2.2 Системный промпт режима оценки (eval-mode)

Используется при тестировании на размеченном датасете отдельных сниппетов кода без привязки к SARIF. Содержит детальные критерии разграничения true/false positive с акцентом на наличие защитных мер.

You are a senior application security engineer performing vulnerability triage.

A SAST scanner has flagged a code snippet as potentially vulnerable.
Your job is to classify it as TRUE POSITIVE (vulnerable) or FALSE POSITIVE (not vulnerable).

Classification criteria - focus on whether DEFENSIVE MEASURES are present:

TRUE POSITIVE (vulnerable):

- Dangerous functions (eval, exec, os.system, pickle.loads, subprocess with shell=True, etc.) are called with user-controlled input and NO protective measures are applied.

FALSE POSITIVE (not vulnerable):

- The code uses a dangerous function BUT applies defensive measures: input validation, sanitization, escaping, allowlisting, type checking, AST restrictions, shell=False, parameterized queries, safe API alternatives, timing-safe comparisons, etc.
- The code only operates on trusted/hardcoded/internal data.

Key principle: "does the code include a protective measure?", NOT "could a skilled attacker bypass it?". If ANY defense exists,

classify as false_positive.

Always respond with valid JSON. No markdown, no code fences.

6.2.3 Структура итогового JSON-ответа

Все промпты требуют от модели возвращать строго формализованный JSON:

```
{  
  "verdict": "true_positive" | "false_positive" | "uncertain",  
  "confidence": <float 0.0-1.0>,  
  "reasoning": "<detailed explanation of your analysis>"  
}
```

Принудительная схема вывода обеспечивает автоматизированную постобработку без необходимости разбора свободного текста и существенно снижает долю невалидных ответов.

Список литературы

- [1] Nguyen, V.-A. ReGVD: Revisiting Graph Neural Networks for Vulnerability Detection / V.-A. Nguyen, D. Q. Nguyen, V. Nguyen, T. Le, Q. H. Tran, D. Phung // Proceedings of the 44th International Conference on Software Engineering Companion (ICSE '22 Companion). — 2022. — P. 164–165. — URL: <https://arxiv.org/abs/2110.07317> (дата обращения: 02.05.2026).
- [2] Дин, Я. Vulnerability Detection with Code Language Models: How Far Are We? / Я. Дин, Я. Фу, О. Ибрахим [и др.] // arXiv preprint. — 2024. — № arXiv:2403.18624. — URL: <https://arxiv.org/abs/2403.18624> (дата обращения: 02.05.2026).
- [3] Фарасат, Т. Machine Learning Techniques for Python Source Code Vulnerability Detection / Т. Фарасат, Й. Посерра // arXiv preprint. — 2024. — № arXiv:2404.09537. — URL: <https://arxiv.org/abs/2404.09537> (дата обращения: 02.05.2026).
- [4] NIST. Risk Management Guide for Information Technology Systems / NIST Special Publication 800-30 Rev. 1. — Gaithersburg: National Institute of Standards and Technology, 2023. — URL: <https://csrc.nist.gov/pubs/sp/800/30/r1/final> (дата обращения: 02.05.2026).
- [5] OWASP Foundation. OWASP Top 10:2024 — The Ten Most Critical Web Application Security Risks / OWASP Foundation. — 2024. — URL: <https://owasp.org/www-project-top-ten/> (дата обращения: 02.05.2026).
- [6] ГОСТ Р 56939-2016. Защита информации. Обеспечение безопасности при разработке программного обеспечения. Термины и определения. — М.: Стандартинформ, 2016. — 12 с.
- [7] MITRE Corporation. Common Weakness Enumeration (CWE) — A Software Community Developed Dictionary of Software Weakness Types / MITRE. — 2025. — URL: <https://cwe.mitre.org> (дата обращения: 02.05.2026).
- [8] FIRST.org. Common Vulnerability Scoring System v4.0: Specification Document / FIRST. — 2023. — URL: <https://www.first.org/cvss/v4.0/specification-document> (дата обращения: 02.05.2026).
- [9] CISA. Known Exploited Vulnerabilities Catalog / Cybersecurity and Infrastructure Security Agency. — 2026. — URL: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog> (дата обращения: 02.05.2026).
- [10] ФСТЭК России. Приказ от 25.12.2017 № 239 «Об утверждении Требований по обеспечению безопасности значимых объектов критической информационной инфраструктуры Российской Федерации». — М., 2017.

- [11] ФСТЭК России. Приказ от 21.12.2020 № 250 «Об утверждении Требований по обеспечению безопасности информации в автоматизированных системах управления производственными и технологическими процессами на критически важных объектах». — М., 2020.
- [12] Seacord, R. C. Secure Coding in C and C++ / R. C. Seacord. — 3rd ed. — Boston: Addison-Wesley Professional, 2021. — 432 p.
- [13] Halfond, W. G. SQL Injection Attacks and Defense / W. G. Halfond, J. Viegas, A. Orso // ACM Computing Surveys. — 2024. — Vol. 56, № 3. — P. 1–35.
- [14] Stuttard, D. The Web Application Hacker’s Handbook: Finding and Exploiting Security Flaws / D. Stuttard, M. Pinto. — 3rd ed. — Indianapolis: Wiley, 2024. — 896 p.
- [15] Chen, X. Use-After-Free Vulnerability Detection via Deep Graph Learning / X. Chen, Y. Li, Z. Wang // IEEE Transactions on Software Engineering. — 2024. — Vol. 50, № 4. — P. 1123–1139.
- [16] Zhou, Y. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks / Y. Zhou, S. Liu, J. Siow, X. Zhang, L. Sun // arXiv preprint. — 2019. — № arXiv:1909.03496. — URL: <https://arxiv.org/abs/1909.03496> (дата обращения: 02.05.2026).
- [17] Guo, D. GraphCodeBERT: Pre-training Code Representations with Data Flow / D. Guo, S. Ren, S. Lu [et al.] // arXiv preprint. — 2021. — № arXiv:2009.08366. — URL: <https://arxiv.org/abs/2009.08366> (дата обращения: 02.05.2026).
- [18] Johnson, J. Billion-scale similarity search with GPUs / J. Johnson, M. Douze, H. Jégou // IEEE Transactions on Big Data. — 2019. — Vol. 7, № 3. — P. 535–547. — URL: <https://arxiv.org/abs/1702.08734> (дата обращения: 02.05.2026).
- [19] Khosla, P. Supervised Contrastive Learning / P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, D. Krishnan // Advances in Neural Information Processing Systems (NeurIPS). — 2020. — Vol. 33. — P. 18661–18673. — URL: <https://arxiv.org/abs/2004.11362> (дата обращения: 02.05.2026).
- [20] Махьяри, А. А. Harnessing the Power of LLMs in Source Code Vulnerability Detection / А. А. Махьяри // arXiv preprint. — 2024. — № arXiv:2408.03489. — URL: <https://arxiv.org/abs/2408.03489> (дата обращения: 02.05.2026).
- [21] Ли, П. Neuro-symbolic Static Analysis with LLM-generated Vulnerability Patterns / П. Ли, С. Яо, Ё. Сарфати Корич [и др.] // arXiv preprint. — 2025. — № arXiv:2504.16057. — URL: <https://arxiv.org/abs/2504.16057> (дата обращения: 02.05.2026).

- [22] Ni, C. MegaVul: A C/C++ Vulnerability Dataset with Comprehensive Code Representations / C. Ni, L. Shen, X. Yang, Y. Zhu, S. Wang // Proceedings of the 21st International Conference on Mining Software Repositories (MSR '24). — 2024. — P. 738–742. — URL: <https://arxiv.org/abs/2406.12415> (дата обращения: 05.05.2026).
- [23] Султана, III. Code Vulnerability Detection: A Comparative Analysis of Emerging Large Language Models / III. Султана, С. Африн, Н. У. Эйсти // arXiv preprint. — 2024. — № arXiv:2409.10490. — URL: <https://arxiv.org/abs/2409.10490> (дата обращения: 02.05.2026).
- [24] Чернов, С. Ю. Методы статического анализа кода в задачах обеспечения информационной безопасности / С. Ю. Чернов, А. В. Петров // Информационная безопасность. — 2023. — Т. 29, № 4. — С. 45–62.
- [25] Cousot, P. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs / P. Cousot, R. Cousot // POPL '77: Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages. — 1977. — P. 238–252.
- [26] SonarSource. SonarQube Documentation: Static Code Analysis for Clean Code / SonarSource. — 2025. — URL: <https://docs.sonarqube.org> (дата обращения: 02.05.2026).
- [27] Checkmarx. The State of Application Security: 2024 Report / Checkmarx Research. — 2024. — URL: <https://www.checkmarx.com/resources/state-of-application-security-report/> (дата обращения: 02.05.2026).
- [28] OpenText. Fortify Static Code Analyzer User Guide / OpenText Security. — 2024. — URL: <https://www.microfocus.com/documentation/fortify> (дата обращения: 02.05.2026).
- [29] GitHub. CodeQL Documentation: Querying Code as Data / GitHub Security Lab. — 2025. — URL: <https://codeql.github.com/docs> (дата обращения: 02.05.2026).
- [30] Return To Corp. Semgrep Documentation: Fast, Open-Source Static Analysis / Semgrep. — 2025. — URL: <https://semgrep.dev/docs> (дата обращения: 02.05.2026).
- [31] PyCQA. Bandit: Security Linter for Python / Python Code Quality Authority. — 2024. — URL: <https://bandit.readthedocs.io> (дата обращения: 02.05.2026).
- [32] Wheeler, D. A. Flawfinder: A Source Code Security Scanner / D. A. Wheeler. — 2023. — URL: <https://www.dwheeler.com/flawfinder/> (дата обращения: 02.05.2026).
- [33] Cppcheck Team. Cppcheck Manual: Static Analysis for C/C++ / Cppcheck. — 2025. — URL: <http://cppcheck.sourceforge.net/manual.pdf> (дата обращения: 02.05.2026).

- [34] Hu, E. J. LoRA: Low-Rank Adaptation of Large Language Models / E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen // arXiv preprint. — 2021. — № arXiv:2106.09685. — URL: <https://arxiv.org/abs/2106.09685> (дата обращения: 02.05.2026).
- [35] Антонов, А. В. Применение LLM и Classical ML алгоритмов для решения задач анализа уязвимостей в коде: исходный код курсовой работы / А. В. Антонов // GitHub. — 2026. — URL: https://github.com/jachant/course_paper_3 (дата обращения: 16.05.2026).